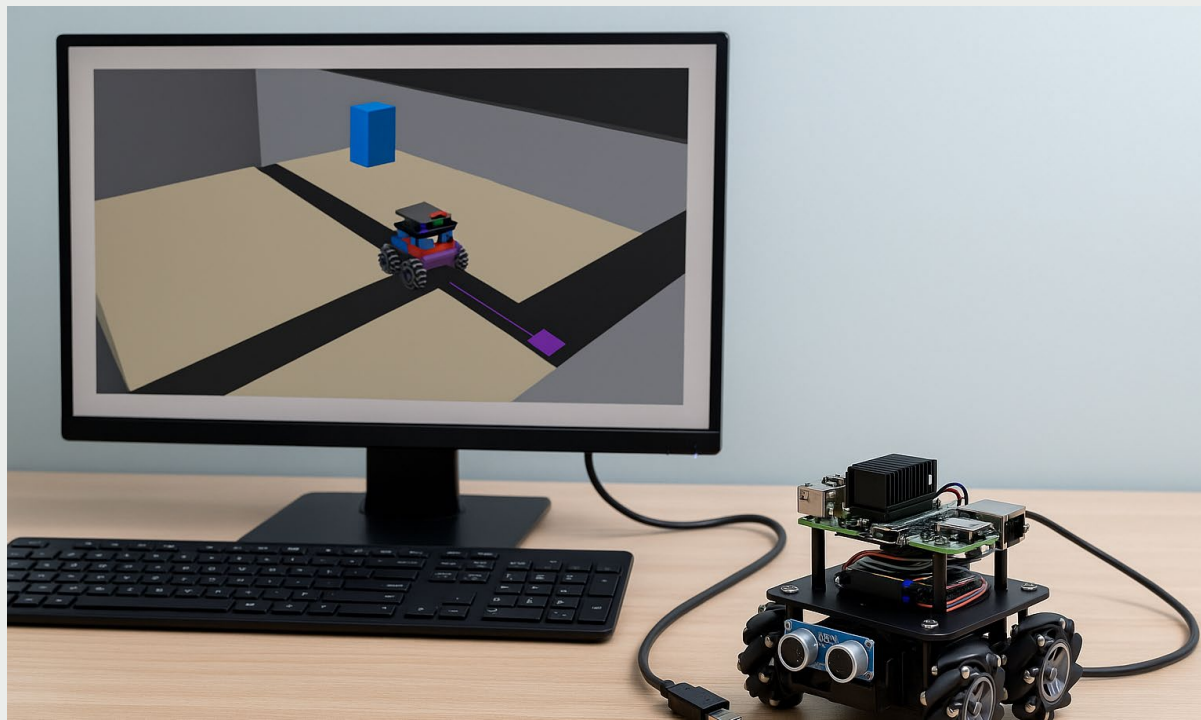


Navegación inteligente en robótica móvil



Mateo Almeida Gómez

Robótica

Prof. Jaime Arango
Castro

17/06/25

Agenda

Revisaremos la robótica móvil desde la importancia de la navegación y demás conceptos teóricos relacionados a esta.

Navegación reactiva

Navegación basada en mapas



Navegación Robótica



¿Qué es la Navegación?

Permite a los robots moverse autónomamente de un punto A a un punto B, evitando obstáculos y alcanzando objetivos.



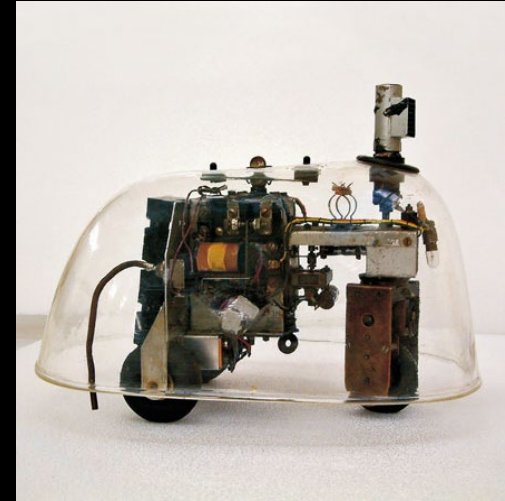
¿Por Qué es Importante?

Es fundamental para aplicaciones como exploración, entrega, servicio y operaciones autónomas en entornos dinámicos.

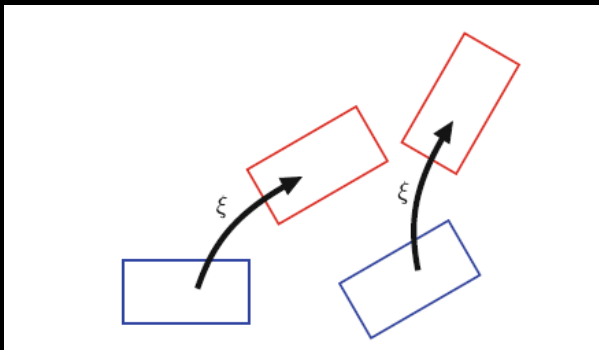
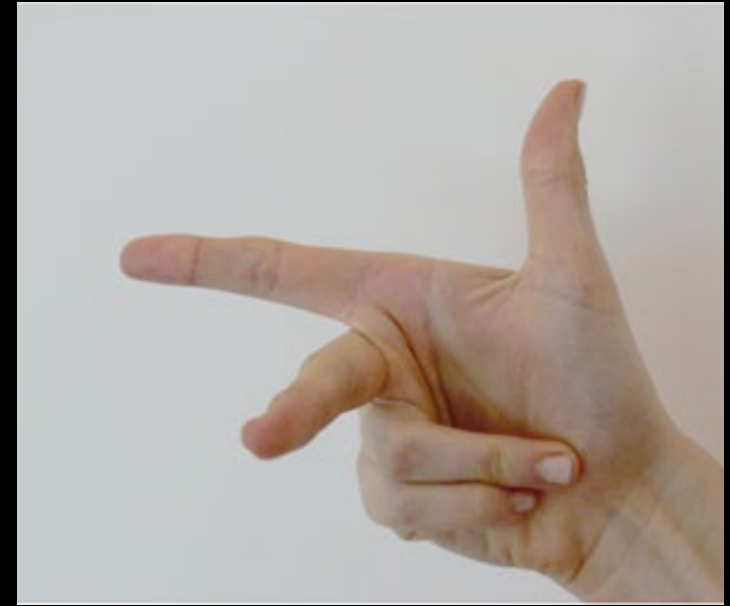


Contexto del Curso

Esta presentación intenta integrar la navegación junto a fundamentos previous previous del libro de Peter Corke.

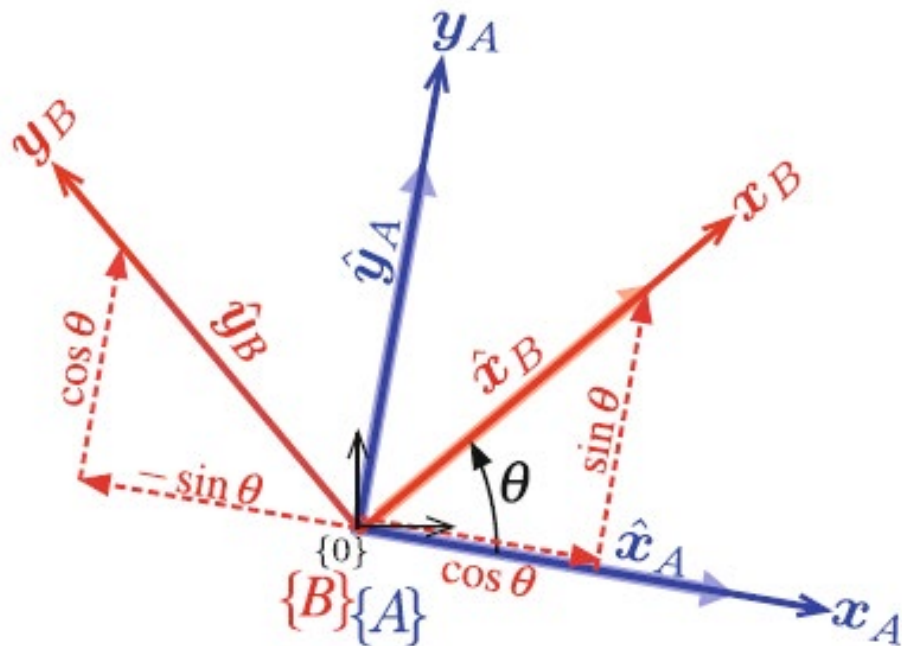
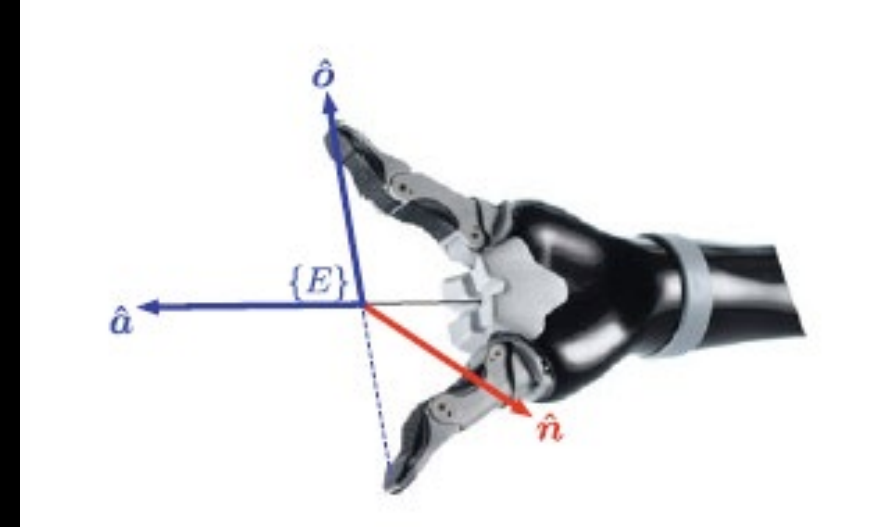


Pose y marcos de coordenadas



Conceptos de pose y marcos de coordenada (representan la posición y orientación de un robot, cámara o los objetos en un entorno que necesita el robot para trabajar)

Posición y orientación (POSE)



Tanto para 2D como para 3D, representamos marcos de coordenadas usando transformaciones que combinan **rotación** y **traslación** con el fin de **representar posición y orientación**.

En el plano, usamos **matrices de rotación 2D**, mientras que en 3D empleamos **matrices de rotación 3x3**, **ángulos de Euler**, **matrices complejas** o **cuaterniones** para describir la orientación de un robot o sensor.

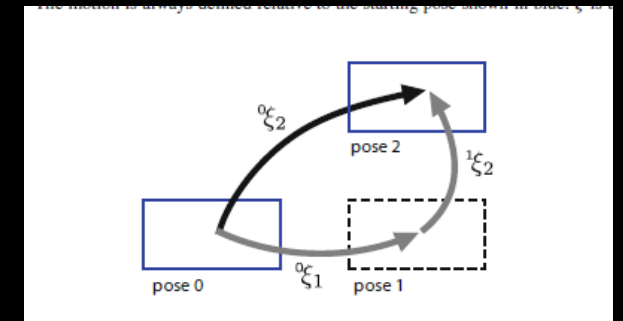
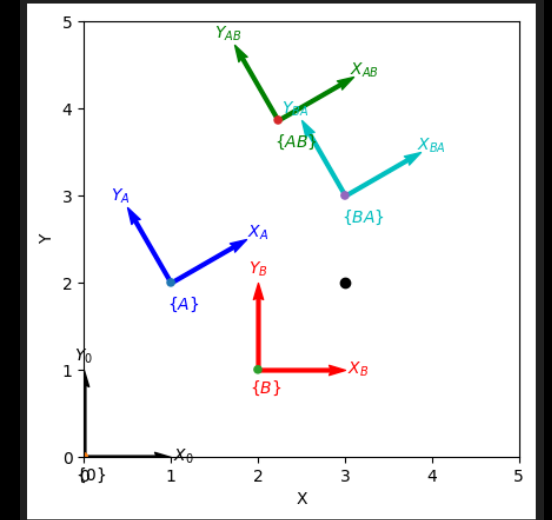
Frames de coordenadas y gráficos de pose

Marcos de coordenadas

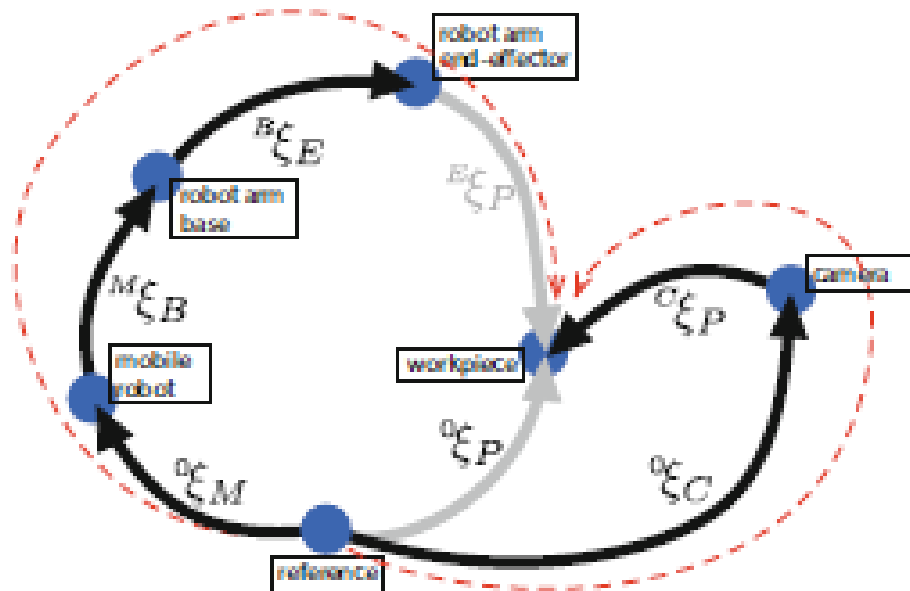
Cada robot o componente (sensor, actuador) define su propio **sistema de coordenadas**.

Poses relativas y gráficos de poses

- La **pose** de un robot es una combinación de su posición y orientación.
- Representan multiples poses relativas entre distintos marcos



GRÁFICOS DE POSES

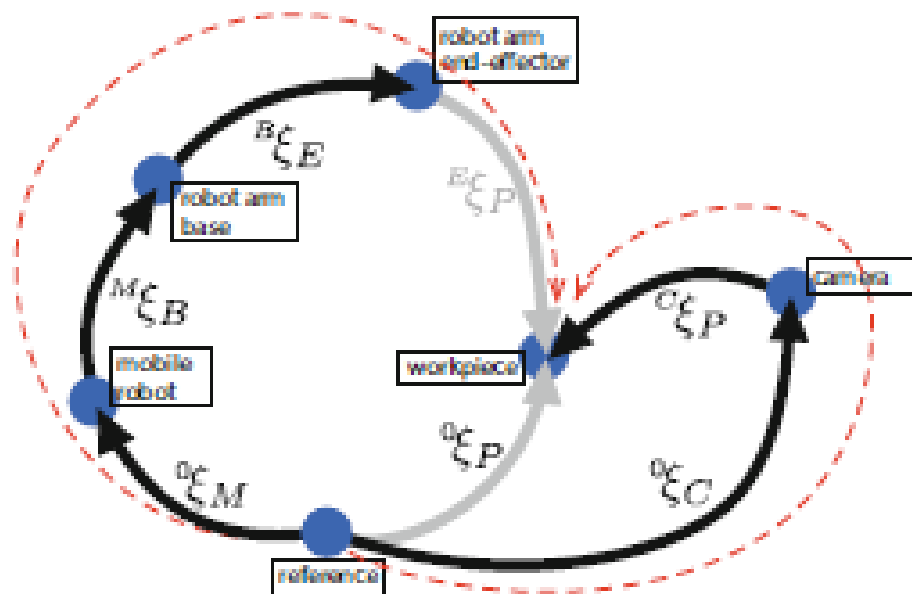


Los nodos representan marcos y las aristas transformaciones relativas.

Permite describir relaciones entre el robot, actuadores y objetos en su entorno

Cada eje representa un marco en el espacio.

ORIENTACIÓN



POSE

2D

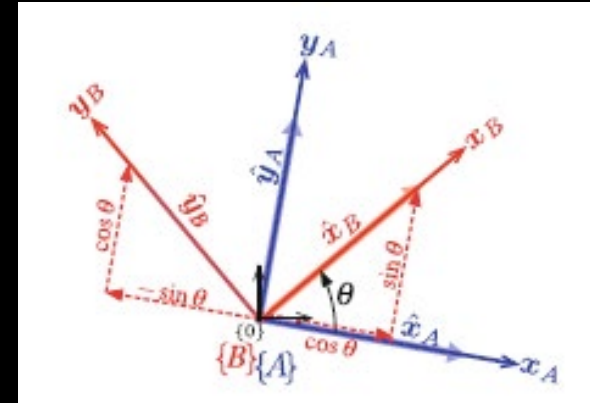
ORIENTACIÓN

MATRIZ DE ROTACIÓN:

```
from sympy import Symbol, Matrix, cos, sin  
  
theta = Symbol('theta')  
R = Matrix([  
    [cos(theta), -sin(theta)],  
    [sin(theta), cos(theta)]  
])
```

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Gira un vector en el plano, alrededor del eje, conservando la longitud y dirección relativa.



Base para representar orientación en 2D y construir transformaciones homogéneas SE(2) –SPECIAL EUCLIDE GROUP

2D

ORIENTACIÓN

MATRIZ EXPONENCIAL PARA ROTACIÓN:

```
linalg.logm(R)
array([[ 0.3, -0.9553],
       [ 0.9553,  0.3]])

linalg.expm(L)
array([[ 0.9553, -0.2955],
       [ 0.2955,  0.9553]])
```

Se puede usar la exponencial de una matriz antisimétrica para generar matrices de rotación

$$R = e^{[\theta]_{\times}} \in SO(2)$$

- Permite generar rotaciones desde velocidad angular.
- Usado en simulación, interpolación y control.

2D

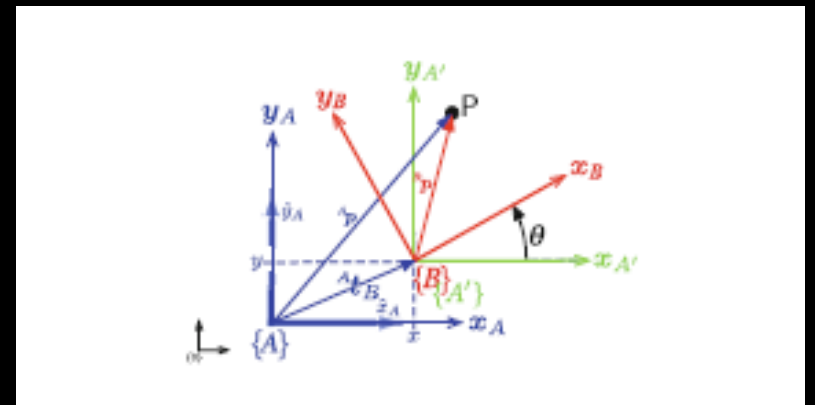
POSE

TRANSFORMACIÓN HOMOGÉNEA

$$\begin{pmatrix} {}^A x \\ {}^A y \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A \mathbf{R}_B(\theta) & {}^A \mathbf{t}_B \\ \mathbf{0}_{1 \times 2} & 1 \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \\ 1 \end{pmatrix}$$

$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \in SE(2)$ Donde t es la traslación y R la orientación del marco B con respecto al marco A

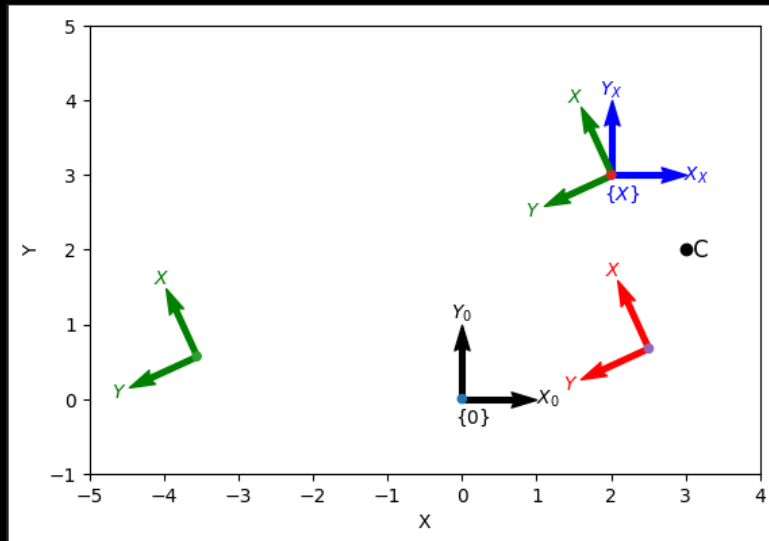
Combina rotación y traslación en un solo paso. Permitiendo Transformar un punto de un marco a otro.



2D

POSE

ROTAR UN MARCO DE COORDENADAS



Aplicar rotación a un sistema de coordenadas, cambia su orientación

Es común transformar puntos de un marco al otro

- Transformación de puntos desde el marco del sensor al marco base.
- Fundamental para tareas como SLAM, control de manipuladores o navegación

2D

POSE

MATRIZ EXPONENCIAL PARA POSE:

Interpolación suave de poses usando matrices exponenciales

$$T = e^{S\theta} \in SE(2)$$

Donde S , es una matriz antisimétrica 3x3

- Animaciones suaves.
- Movimiento continuo entre dos configuraciones.
- Planeación de trayectorias en robots móviles o brazos robóticos

2D

POSE

```
S = Twist2.UnitRevolute(C)
✓ 0.0s
(2 -3; 1)

linalg.expm(skewa(2 * S.S))
✓ 0.0s
array([[ -0.4161,  -0.9093,   6.067],
       [  0.9093,  -0.4161,   0.1044],
       [         0,         0,         1]])

S.exp(2)
✓ 0.0s
-0.4161  -0.9093  6.067
0.9093  -0.4161  0.1044
0         0         1

S = Twist2(T)
✓ 0.0s
(3.9372 3.1663; 0.5)

S.w
✓ 0.0s
0.5

S.pole
✓ 0.0s
array([ -3.166,   3.937])

S.exp(1)
✓ 0.0s
0.8776  -0.4794  3
0.4794   0.8776  4
0         0         1
```

Representan velocidades de movimiento en el plano.

La función twist representa un giro 2D como un vector (v, w) , donde v es traslación y w rotación.

A partir de este, se genera una transformación homogénea aplicando escalado y exponenciación, con un punto fijo s como centro de rotación.

3D

ORIENTACIÓN

MATRIZ DE ROTACIÓN:

$$\begin{pmatrix} {}^A p_x \\ {}^A p_y \\ {}^A p_z \end{pmatrix} = {}^A \mathbf{R}_B \begin{pmatrix} {}^B p_x \\ {}^B p_y \\ {}^B p_z \end{pmatrix}$$

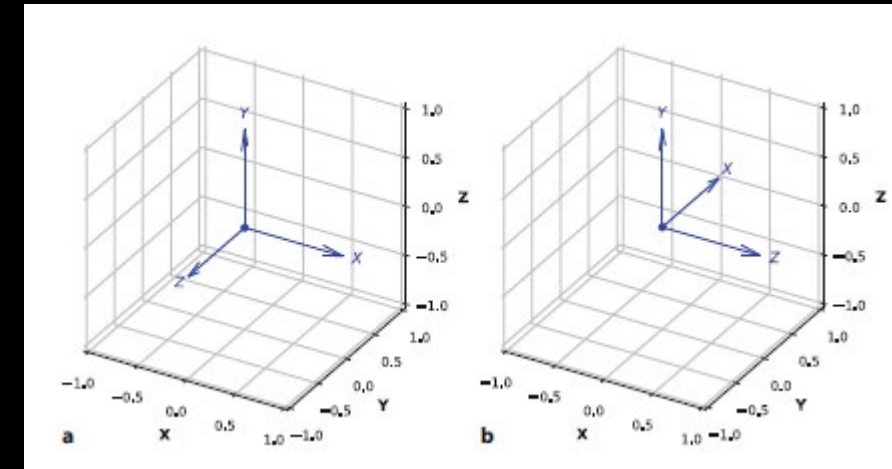
```
from spatialmath.base.transforms3d import rotx, roty
from math import pi

R = rotx(pi / 2)    # rotation about x-axis
#R = rotx(pi / 2) @ roty(pi / 2)    # rotation about z-axis
✓ 0.0s
```

```
array([[ 1,  0,  0],
       [ 0,  0, -1],
       [ 0,  1,  0]])
```

```
plotvol3(new=True) # for matplotlib/widget
trplot(R);
✓ 0.2s
```

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}$$
$$\mathbf{R}_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$



- “Ortogonal 3×3, det=1.
- Prevenir bloqueo de cardan
- Se pueden combinar generando rotaciones compuestas.

3D

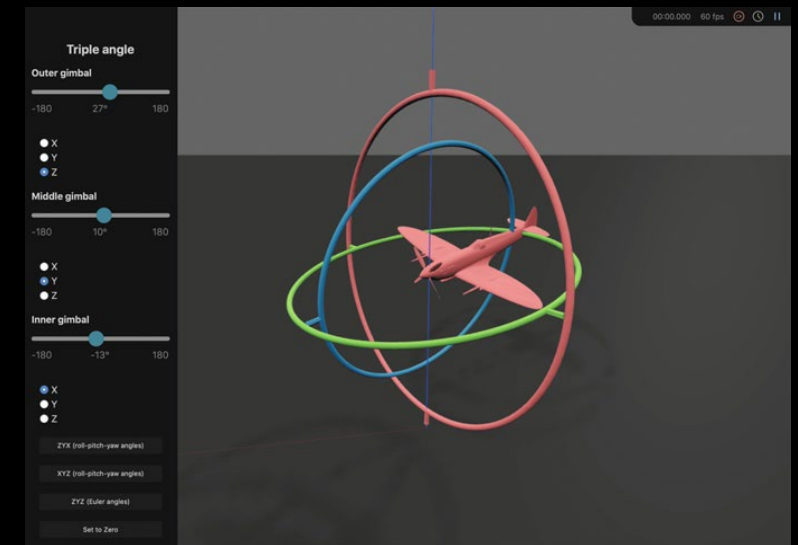
ORIENTACIÓN

Three angle representation

El teorema de rotación de Euler significa que una rotación entre dos sistemas de coordenadas cualesquiera en 3D puede representarse mediante una secuencia de tres ángulos de rotación

También se puede representar con los **Ángulos de Tait-Bryan (o RPY)**, son rotaciones alrededor de **tres ejes diferentes**, típicamente Z-Y-X, también conocidos como **yaw-pitch-roll**

Finalmente se combinan tres rotaciones sucesivas al rededor de los ejes.



3D

ORIENTACIÓN

SINGULARIDADES

el Apollo 11 casi falla en su alunizaje por un problema matemático llamado gimbal lock

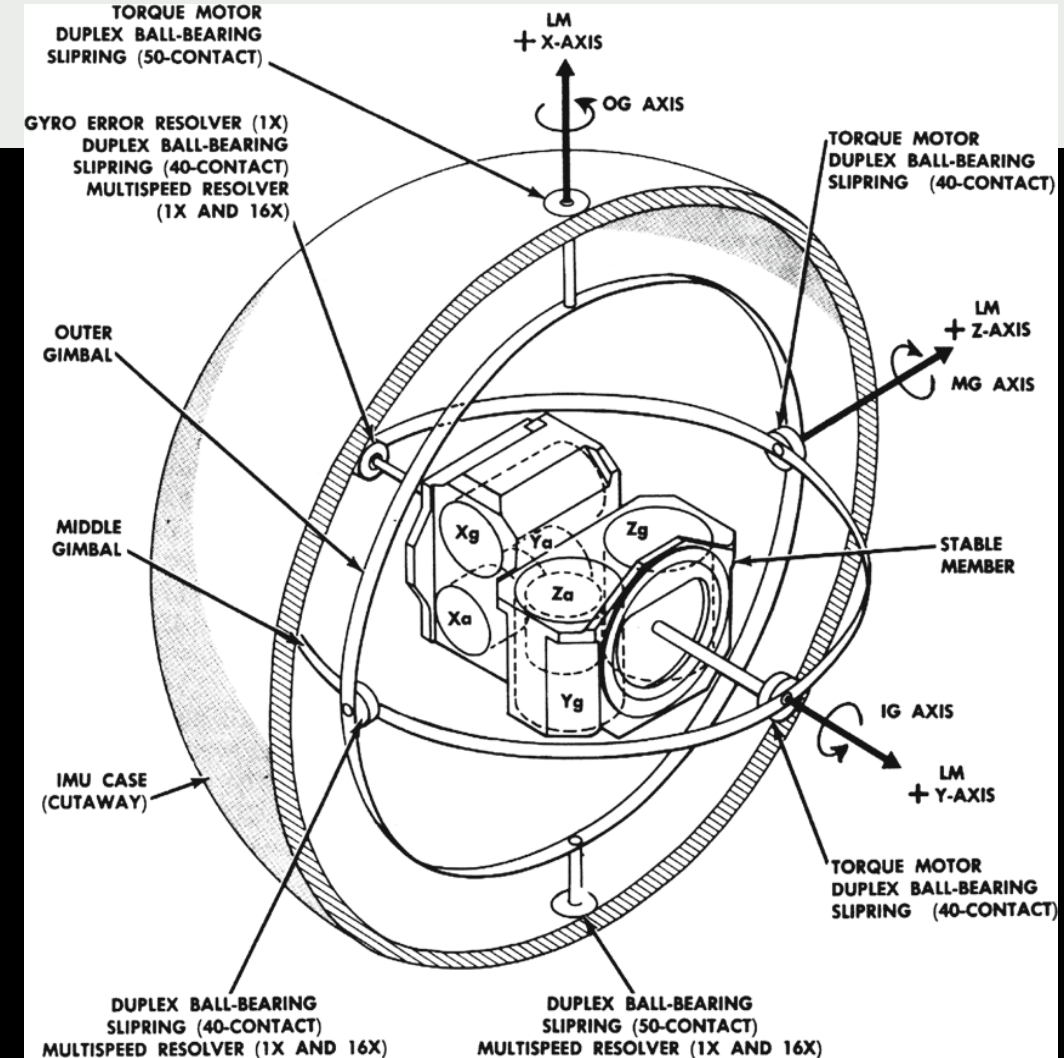
Ocurre cuando dos ejes de rotación se alinean, y es un riesgo real en robótica cuando usamos ángulos de Euler.

Esto provoca :

Fallos en navegación (drones ,IMUS)

Limitaciones en brazos robóticos

Errores en simuladores de realidad virtual



3D

ORIENTACIÓN

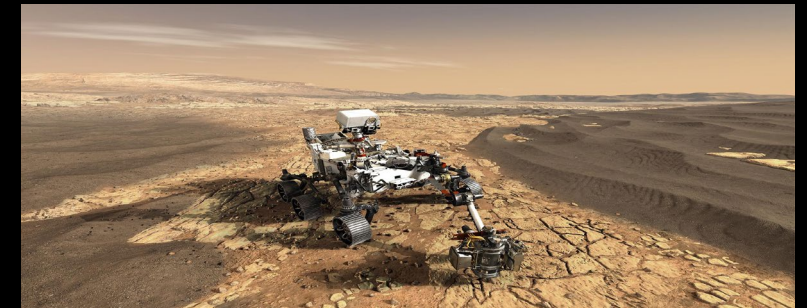
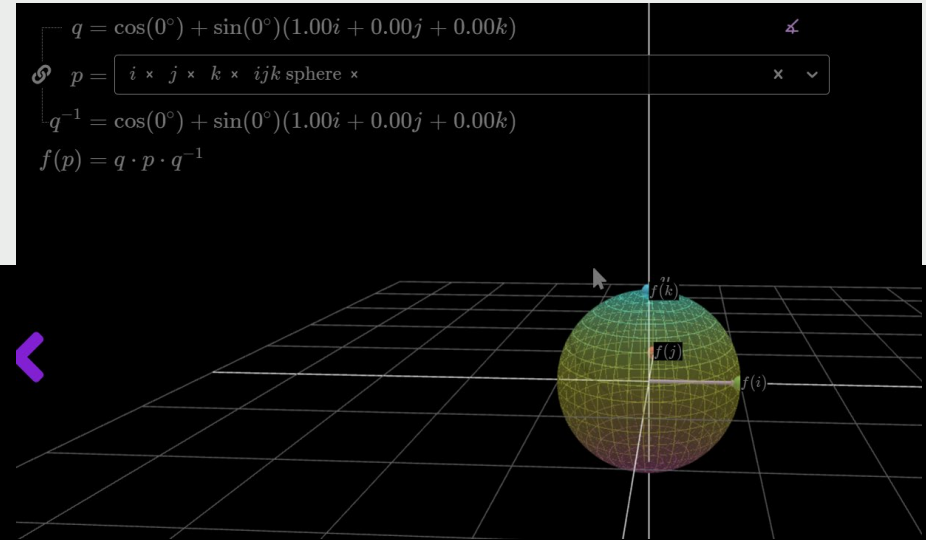
CUATERNIONES

$$\check{q} = s + v_x i + v_y j + v_z k \in \mathbb{H}$$

- Presenta una formula de rotación:

$$p' = q p q^{-1}.$$

– Para evitar el gimbal lock (de tres ejes de rotación secuenciales, dos ejes se alinean y se pierde un grado de libertad) usamos **Cuaterniones** o **matrices de rotación**: Estas son representaciones que no tienen esta singularidad y permiten una interpolación suave entre orientaciones (SLERP = interpolación esférica de cuaterniones)



3D

ORIENTACIÓN

MATRIX EXPONENCIAL PARA ROTACIÓN

- Similar a 2D pero en $SO(3)$ usando matrices Antisimétricas y exponenciales, permitiendo interpolar rotaciones

Se genera una rotación en el eje x con $\text{rot}(x)$, luego aplicamos logm para obtener su forma logarítmica, ie, una matriz antisimétrica.

Se representa una rotación de 0.3 alrededor del eje x

$$R = e^{\theta [w]} \in SO(3)$$

Donde theta es el ángulo y w, la matriz antisimétrica del eje de rotación.

```
R = rotx(0.3);
```

✓ 0.0s

```
R = linalg.expm(0.3 * skew([1, 0, 0]));
```

✓ 0.0s

```
X = skew([1, 2, 3])
```

✓ 0.0s

```
array([[ 0, -3,  2],  
       [ 3,  0, -1],  
       [-2,  1,  0]])
```

3D

POSE

TRANSFORMACIÓN HOMOGÉNEA

$$\begin{pmatrix} A_x \\ A_y \\ A_z \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A\mathbf{R}_B & {}^A\mathbf{t}_B \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} \begin{pmatrix} B_x \\ B_y \\ B_z \\ 1 \end{pmatrix}$$

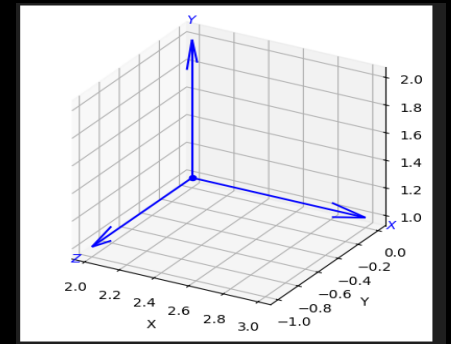
$$; T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Transformación rigida y paralela de rotación y traslación

```
T = transl(2, 0, 0) @ trotx(pi / 2) @ transl(0, 1, 0)
✓ 0.0s

array([[ 1,  0,  0,  2],
       [ 0,  0, -1,  0],
       [ 0,  1,  0,  1],
       [ 0,  0,  0,  1]])

plotvol3(new=True) # for matplotlib/widget
trplot(T);
✓ 0.2s
```



Esto representa trasladarse 2 unidades sobre el eje x, rotar 90° sobre el eje x, por ultimo trasladarse 1 unidad sobre el nuevo eje y.

Como funciones claves tenemos `t2r(T)` extrae la parte de rotación (3×3) y `transl(T)` extrae la parte de traslación (3×1).

3D

POSE

MATRIZ EXPONENCIAL PARA POSE:

Para este caso se usa exponencial de matrices con el fin de interpolar poses

$$T = e^{[S]} \in SE(3)$$

where $S \in \mathbb{R}^6$ and $[\cdot] : \mathbb{R}^6 \mapsto se(3) \subset \mathbb{R}^{4 \times 4}$.

Donde S , es un vector girado

3D

POSE

Representa velocidad lineal
y angular del cuerpo.

```
S = Twist3.UnitRevolute([1, 0, 0], [0, 0, 0])
✓ 0.0s
(0 0 0; 1 0 0)

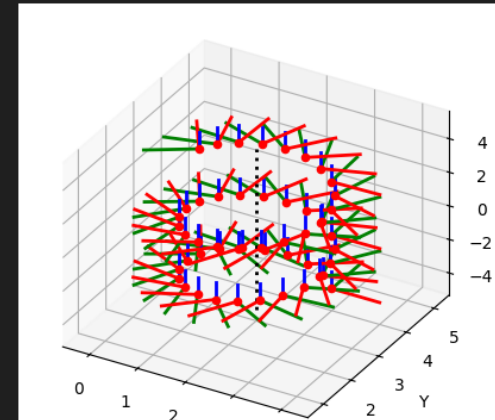
linalg.expm(0.3 * skewa(S.S)); # different to book, see §2.2.1.2
✓ 0.0s

S.exp(0.3)
✓ 0.0s
1      0      0      0
0      0.9553 -0.2955 0
0      0.2955 0.9553 0
0      0      0      1

S = Twist3.UnitRevolute([0, 0, 1], [2, 3, 2], 0.5)
✓ 0.0s
(3 -2 0.5; 0 0 1)
```

```
for theta in np.arange(0, 15, 0.3):
    trplot(S.exp(theta).A @ X, style="rviz", width=2)

L = S.line()
L.plot('k:', linewidth=2);
✓ 1.0s
Axes3D(0.125,0.11;0.775x0.77)
```



3D

POSE

VECTOR CUATERNIÓN

$$(t, \overset{\circ}{q}) \in \mathbb{R}^3 \times S^3.$$

composition	$\xi_1 \oplus \xi_2$	$\mapsto (t_1 + \overset{\circ}{q}_1 \cdot t_2, \overset{\circ}{q}_1 \circ \overset{\circ}{q}_2)$
inverse	$\ominus \xi$	$\mapsto (-\overset{\circ}{q}^* \cdot t, \overset{\circ}{q}^*)$
identity	$\emptyset$	$\mapsto (\mathbf{0}_3, 1\langle 0 \rangle)$
vector-transform	$\xi \cdot v$	$\mapsto \overset{\circ}{q} \cdot v + t$

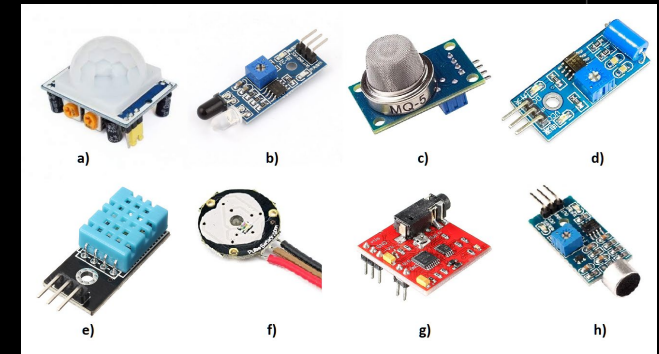
CUATERNIÓN DUAL UNITARIO

Rotation and translation in 3D can be implemented by a unit dual quaternion $(\overset{\circ}{q}_r, \check{q}_d) \in S^3 \times \mathbb{H}$ which comprises a unit quaternion $\overset{\circ}{q}_r$

composition	$\xi_1 \oplus \xi_2$	$\mapsto (\overset{\circ}{q}_{r1} \circ \overset{\circ}{q}_{r2}, \overset{\circ}{q}_{r1} \circ \check{q}_{d2} + \check{q}_{d1} \circ \overset{\circ}{q}_{r2})$
inverse	$\ominus \xi$	$\mapsto (\overset{\circ}{q}_r, \check{q}_d)^* = (\overset{\circ}{q}_r^*, \check{q}_d^*),$ conjugation
identity	$\emptyset$	$\mapsto (1\langle 0 \rangle, 1\langle 0 \rangle)$
vector-transform	$\xi \cdot v$	$\mapsto (\overset{\circ}{q}_r, \check{q}_d) \circ (1\langle 0 \rangle, \check{v}) \circ (\overset{\circ}{q}_r, \check{q}_d)^*,$ where the middle term is a pure dual quaternion.

Cambios entre marcos de referencia (base → herramienta). APLICANDO TRANSFORMACIONES HOMOGENEAS.

Transformación de sensores (IMU, LIDAR, cámaras).
Donde los sensores deben alinearse al marco del robot para interpretar datos correctamente.



Relación con posición, orientación y tiempo

TIEMPO Y MOVIMIENTO

POSE QUE VARÍA EN EL TIEMPO

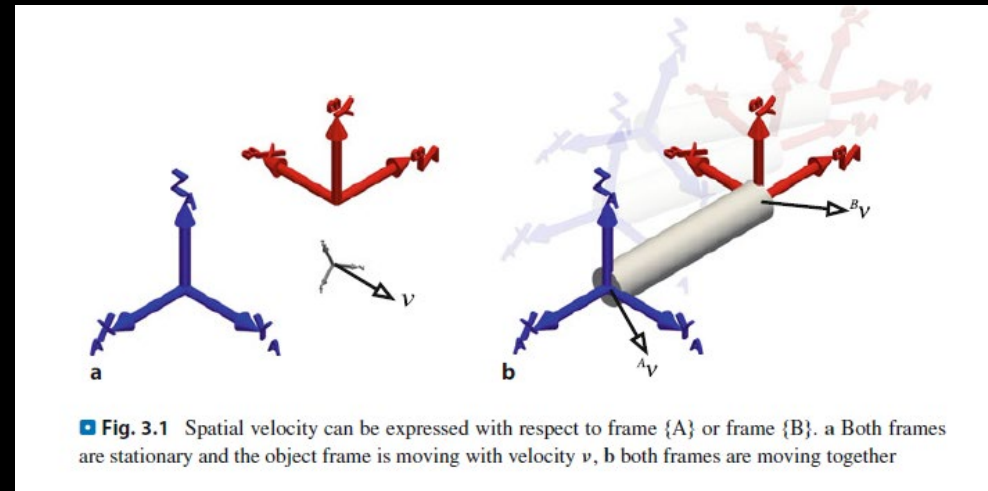
Taza de cambio de orientación y pose

$${}^A\dot{\mathbf{R}}_B = [{}^A\boldsymbol{\omega}_B]_{\times} {}^A\mathbf{R}_B \in \mathbb{R}^{3 \times 3}$$

ie, posición en función del tiempo: $x(t), y(t), \theta(t)$.
Cuya posición + orientación de un marco
varian en el tiempo.

Transformar velocidades espaciales

$${}^A\dot{\mathbf{T}}_B = \begin{pmatrix} {}^A\dot{\mathbf{R}}_B & {}^A\dot{\mathbf{t}}_B \\ \mathbf{0}_{1 \times 3} & 0 \end{pmatrix} = \begin{pmatrix} [{}^A\boldsymbol{\omega}_B]_{\times} {}^A\mathbf{R}_B & {}^A\dot{\mathbf{t}}_B \\ \mathbf{0}_{1 \times 3} & 0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}.$$



Util para navegación, visualización en tiempo real en robots diferenciales U omnidireccionales

TIEMPO Y MOVIMIENTO

CUERPOS ACELERADOS Y MARCOS DE REFERENCIA

NEWTON Y TORQUES ROTACIONALES.

Leyes aplicadas a robots moviles:

$$F = ma, \tau = I\alpha$$

Las fuerzas y torques deben transformarse entre marcos

IMPORTANCIA DEL MARCO INERCIAL EN SENSORES (imu) : Referencia para aplicar Leyes físicas y hacer una estimación de pose mediante la integracion de la aceleración y velocidad angular

IMU ASUME estar en un marco casi inercial, lo que hace sensible a aceleraciones reales



Newton (1642–1727)

TIEMPO Y MOVIMIENTO

POSES VARIABLES EN EL TIEMPO

INTERPOLACION TEMPORAL Y TRAYECTORIAS SUAVES

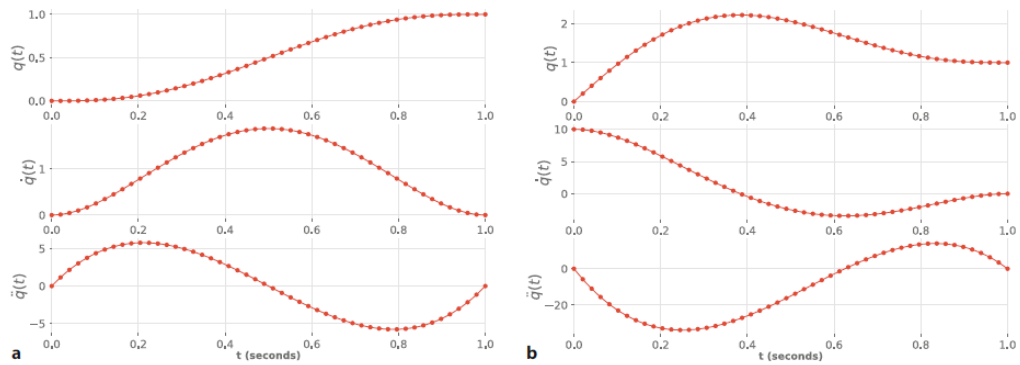


Fig. 3.2 Quintic polynomial trajectory. From top to bottom is position, velocity and acceleration versus time step. a With zero initial and final velocity, b initial velocity of 10 and a final velocity of 0. The discrete-time points are indicated by dots

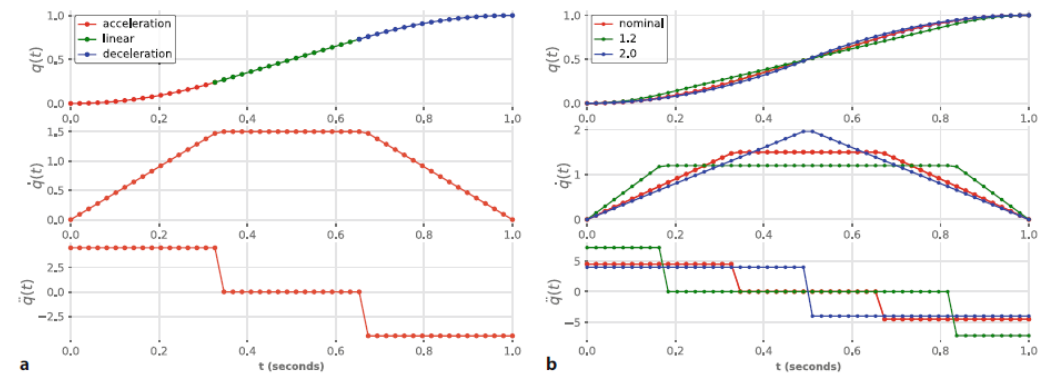
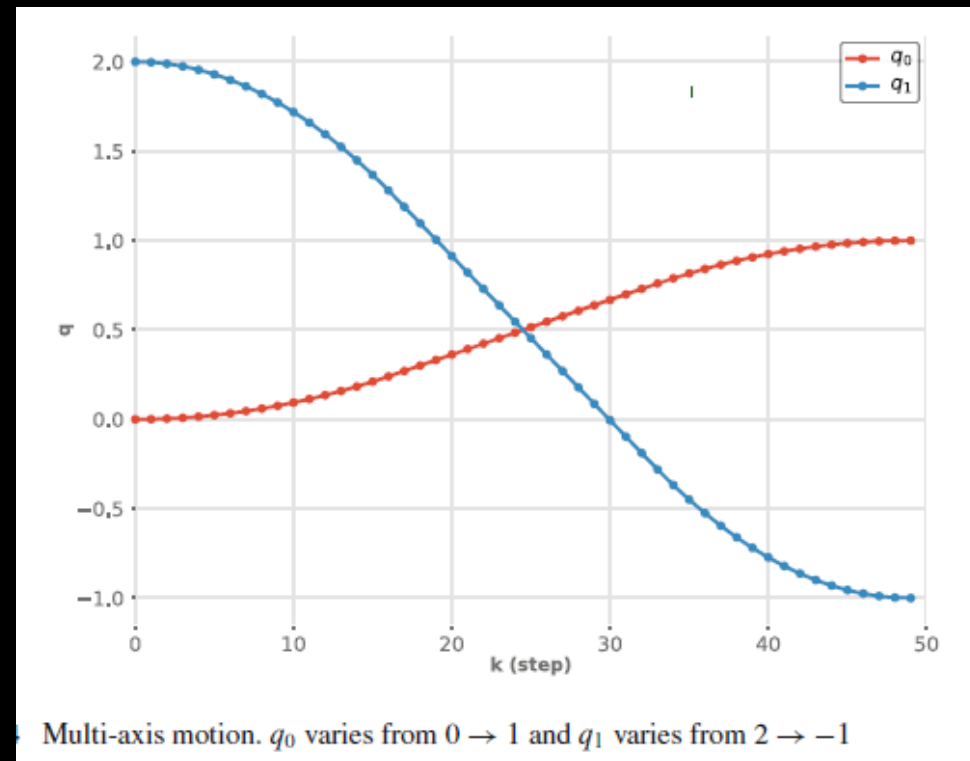


Fig. 3.3 Trapezoidal trajectory. a trajectory for default linear segment velocity; b trajectories for specified linear segment velocities

TIEMPO Y MOVIMIENTO

POSES VARIABLES EN EL TIEMPO

TRAYECTORIAS MULTI EJES

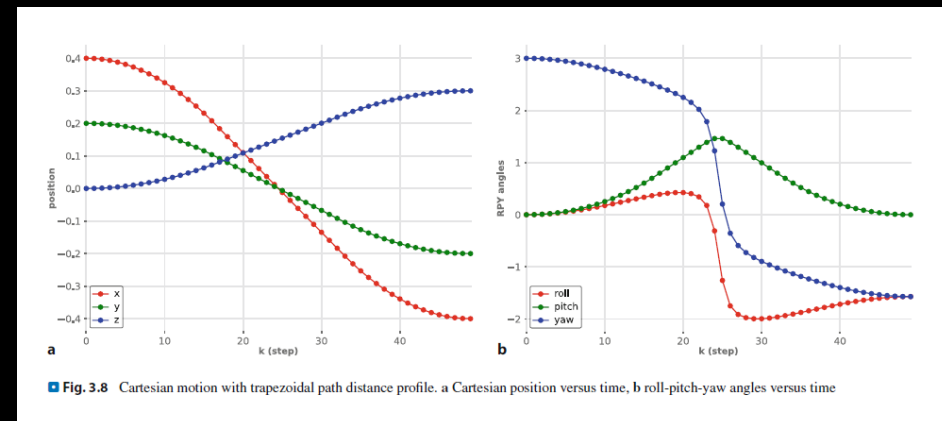
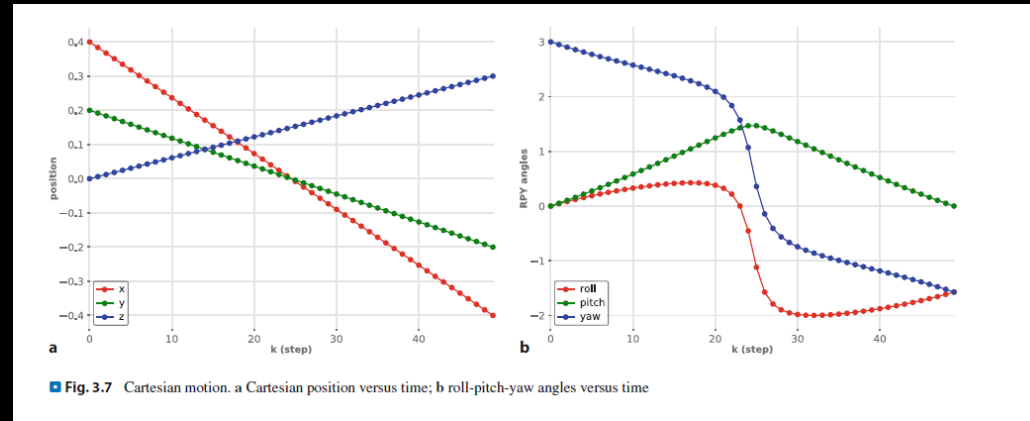


TIEMPO Y MOVIMIENTO

POSES VARIABLES EN EL TIEMPO

INTERPOLACIÓN DE ORIENTACIÓN (SLERP)

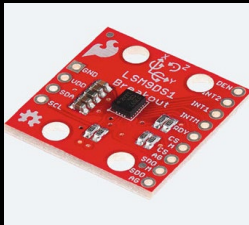
MOVIMIENTO CARTESIANO EN 3D



TIEMPO Y MOVIMIENTO

NAVEGACIÓN INERCIAL

IMU



ULTRASÓNICO : ACCELERÓMETRO Y GIROSCOPIO.

Cómo estiman orientación, pose y aceleración del cuerpo.

TÉCNICA DE DEAD RECKONING y acumulación DE ERRORES

Importante como sistema de apoyo para el SLAM :
proporciona odometría relativa

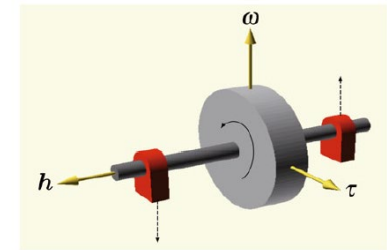
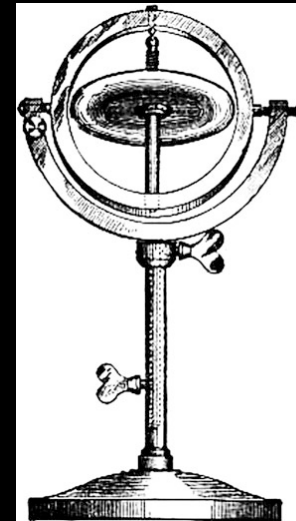


Fig. 3.10 Gyroscope in strapdown configuration. Angular velocity ω induces a torque τ which can be sensed as forces at the bearings shown in red

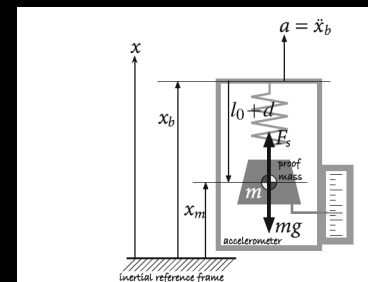


Fig. 3.12 The essential elements of an accelerometer and notation

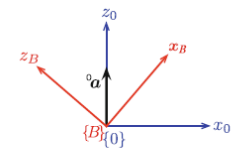


Fig. 3.13 Gravitational acceleration is defined as the z -axis of the fixed frame $\{0\}$ and is measured by an accelerometer in the rotated body frame $\{B\}$

Transformaciones Homogéneas Homogéneas	Representan posición y orientación en 3D.	Esenciales para la localización del robot y la transformación de sensores.
Cuaterniones vs. Euler	Métodos para representar rotaciones.	Los cuaterniones evitan el bloqueo de cardán, crucial para la orientación estable del robot.
Trayectorias	Definen el camino temporal del del robot (polinomios, interpolación).	La planificación de trayectoria asegura movimientos suaves y seguros.
Velocidad y Aceleración	Dinámica del movimiento del robot.	Limitaciones de velocidad y aceleración deben considerarse para movimientos controlados.

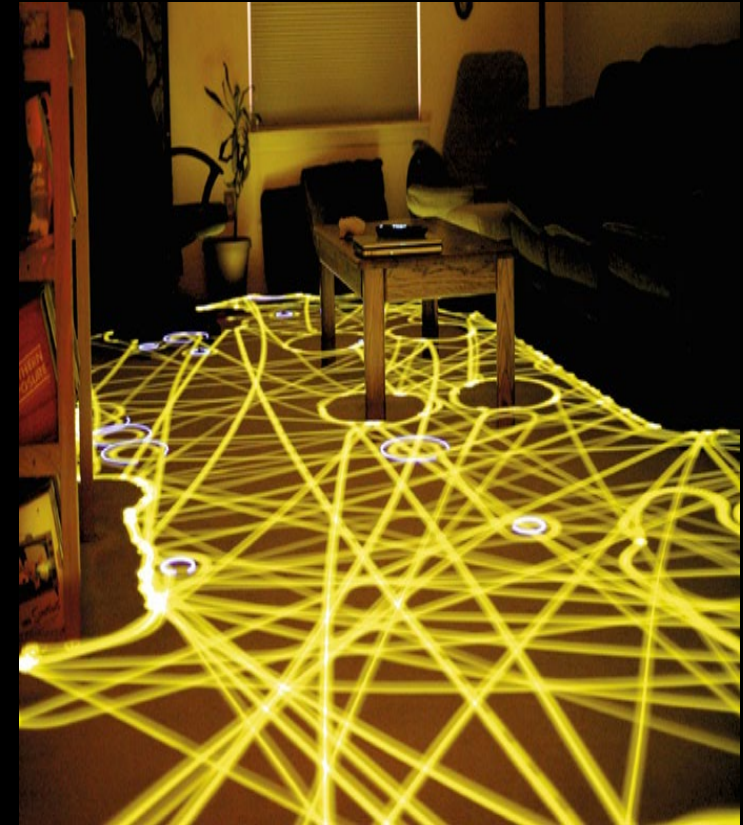
Posición, orientación y tiempo

Tipos de Navegación

Navegación Reactiva (Local)

Responde a la información del sensor inmediato para evitar colisiones sin conocimiento global del entorno.

- Basada en reglas simples.
- Ideal para entornos dinámicos.
- Observamos en pantalla el lapso de tiempo en el cual se encuentra emergido el robot de navegacion reactiva



Navegación Reactiva: Braitenberg y Bug2

Vehículos Braitenberg

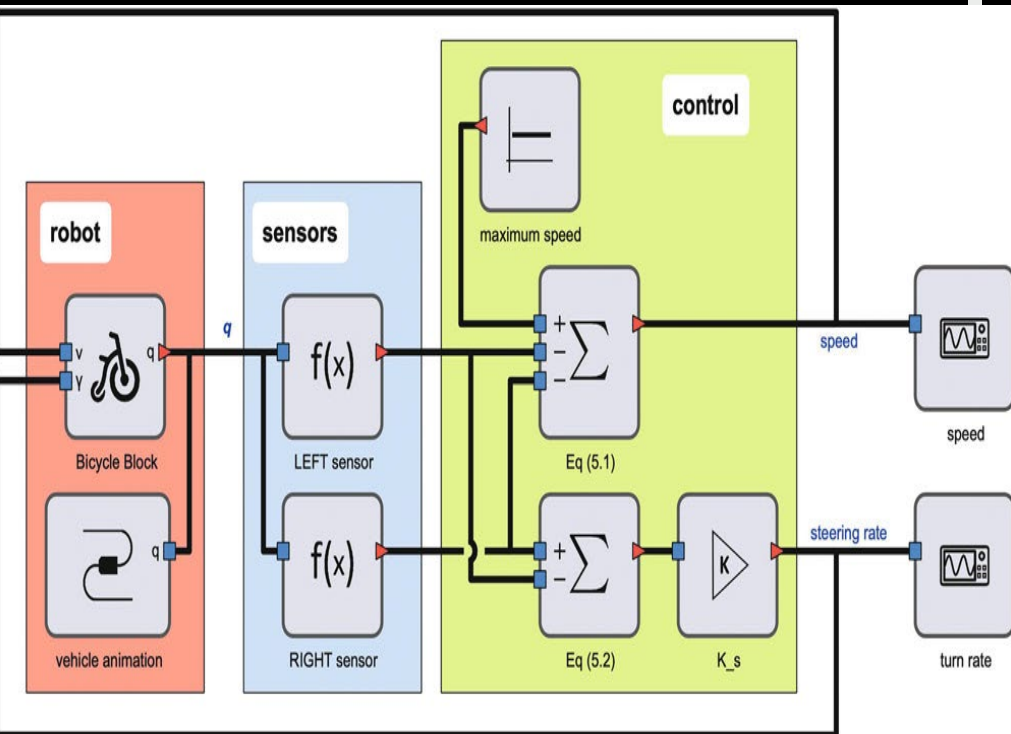
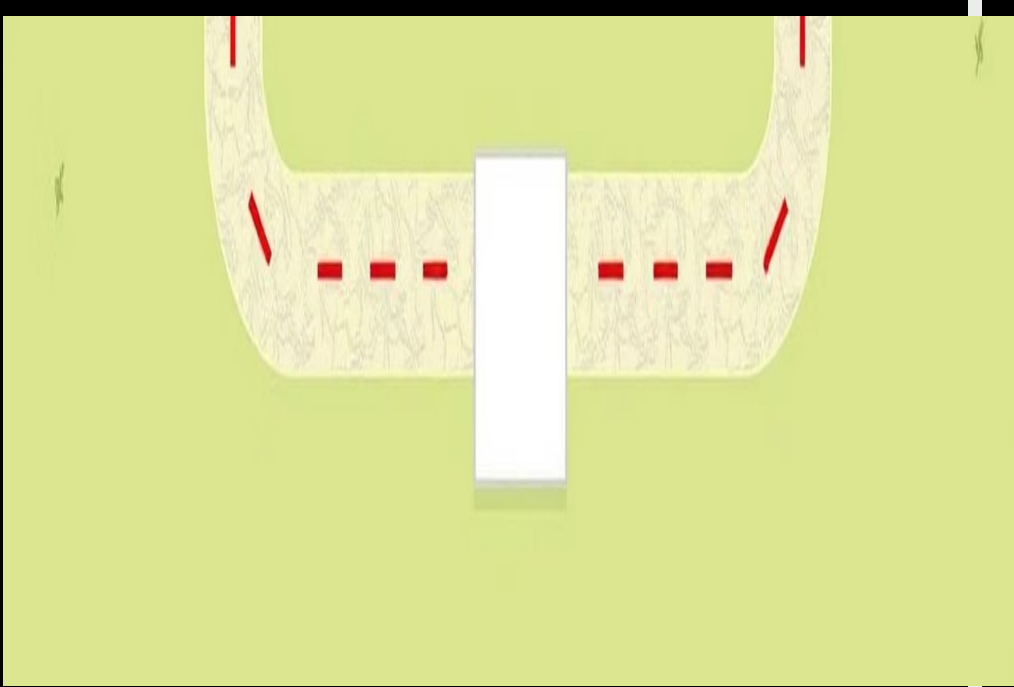
Conexión directa sensor-
actuador; el robot responde a
a estímulos simples sin
planificación compleja
Su conexión sensorial actuadora
actuadora es un plan implícito
implícito

Ventajas y Desventajas

Rápida y simple, pero subóptima y sin previsión, pudiendo
quedar atrapado.

Algoritmo Bug2

Combina avance directo hacia la
meta con rodeo de obstáculos. Sigue
la línea m y se adhiere a contornos.



Braitenberg

```
def sensorfield(x, y):  
    xc, yc = (60, 90)  
    return 200 / ((x - xc) ** 2 + (y - yc) ** 2 + 200)
```

Python

Conexión directa sensor-
actuador; el robot responde a
estímulos simples sin
planificación compleja

Su conexión sensorial actuadora
es un plan implícito

Esta función representa un
campo escalar generado por un
sensor (o una fuente) que está
ubicado en el punto (60, 90). El
campo disminuye con la
distancia desde ese
punto.

De esta forma, el robot tendrá su
velocidad y ángulo de giro,
definidos

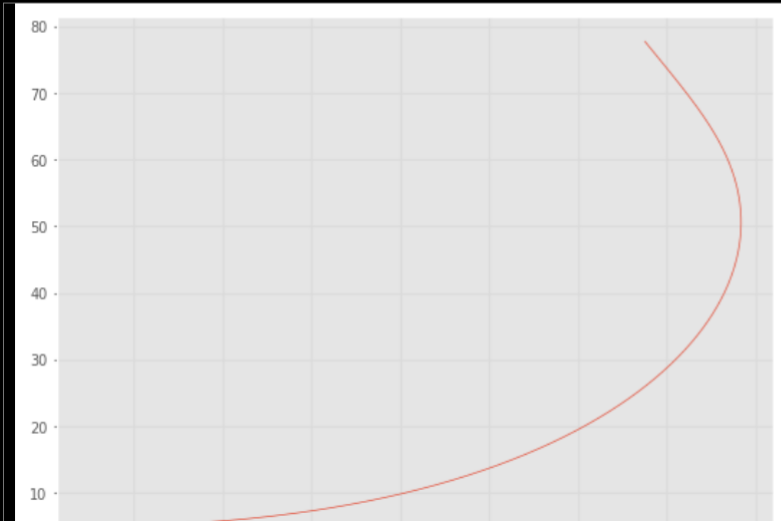
La gráfica muestra la trayectoria del vehículo Braitenberg en
el plano XY, indicando cómo se mueve desde su posición
inicial hacia el máximo del campo sensorial

python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages

```
plt.plot(out.x[:,0], out.x[:,1]);
```

✓ 0.6s

Pytho



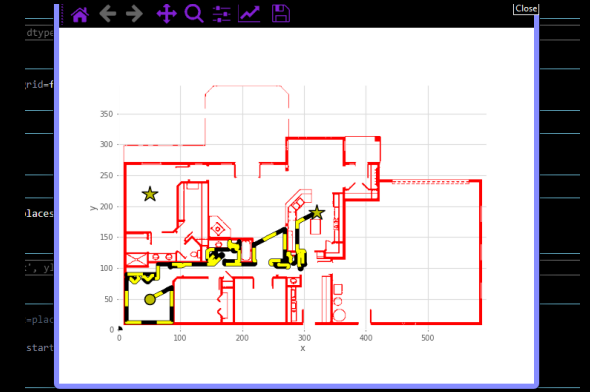
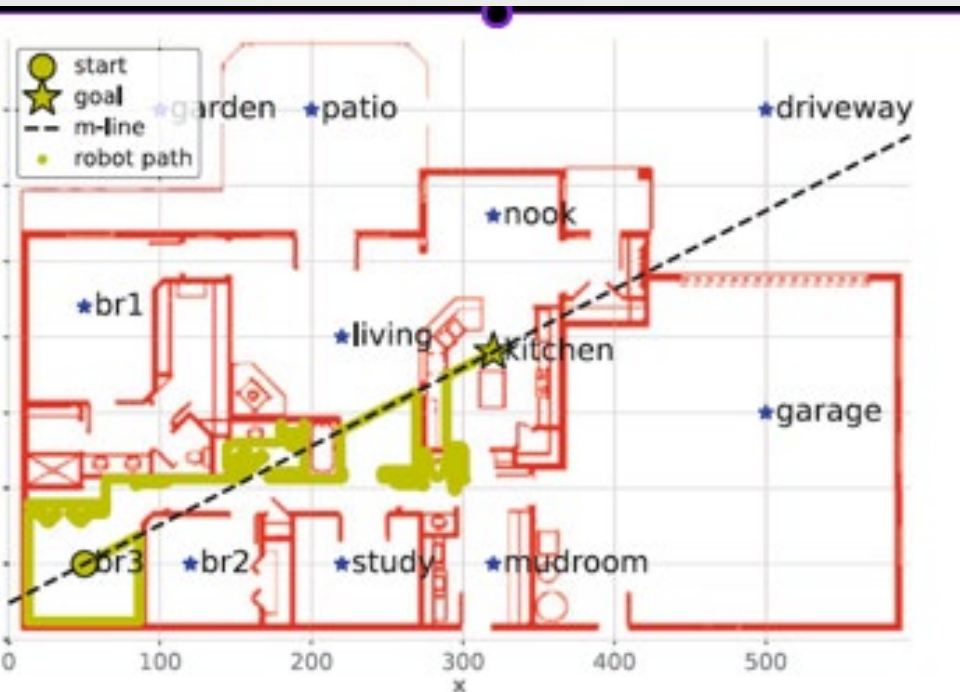
BUGs algorithms

SIMPLE AUTOMATA

A diferencia de los vehículos de Braitenberg, los "bugs" **tienen una memoria mínima** y una **lógica de control más compleja**, por ejemplo, máquinas de estados.

El proceso es sencillo:

1. Dibuja una línea recta (llamada "m-line") desde el punto de inicio al objetivo.
2. El robot se intenta mover a lo largo de la línea
3. Si se encuentra un obstáculo, lo rodea por la derecha, siempre manteniéndose cerca del borde
4. Cuando Vuelve a intersectar la m-line, en un punto más cercano al objetivo que el anterior, regresa a esta y sigue avanzando hacia el objetivo.
5. Repite el proceso hasta llegar al destino.



BUGs algorithms

SIMPLE AUTOMATA

De esta manera, vemos ;

El algoritmo no usa un mapa global, por lo que:

Puede tomar rutas subóptimas como pasar por armarios o baños

Toma decisiones locales sin saber si son contundentes a largo plazo

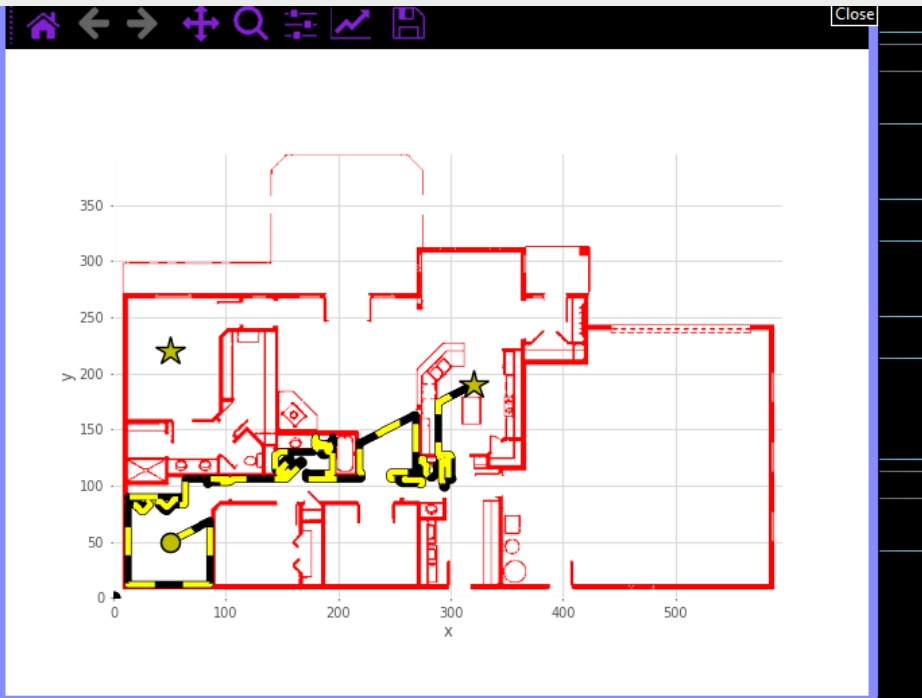
Esto es bueno puesto; que entrega simplicidad y no necesita un mapa completo pero también es malo, debido a que puede dar vueltas innecesarias y no garantiza la mejor ruta

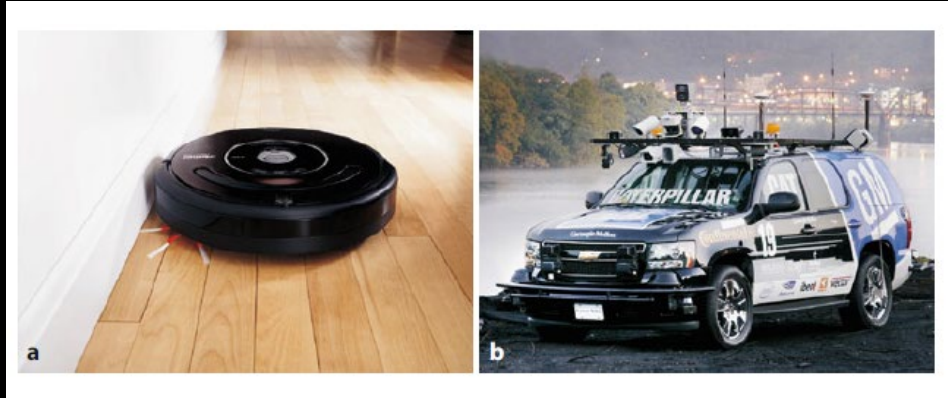
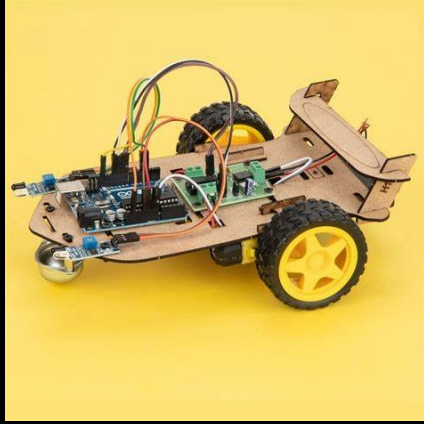
Bug2 ilustra cómo un robot reactivo puede llegar a su meta sin un mapa previo, pero a costa de rutas más largas y menos eficientes. Este algoritmo muestra:

Cómo diseñar comportamientos sencillos basados únicamente en lecturas de sensores.

Las limitaciones que surgen al carecer de visión global del entorno.

El funcionamiento de las estrategias de navegación puramente locales.





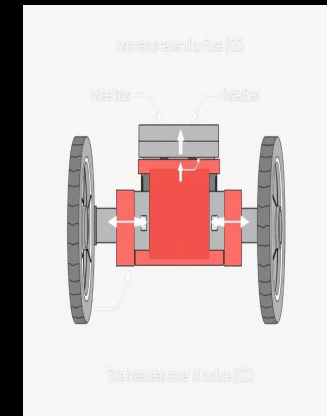
ROBOTS CON RUEDAS

TIPOS DE ROBOTS MOVILES

MODELOS CINEMÁTICOS

La navegación se basa en la comprensión del movimiento del robot en un plano. Las ecuaciones cinemáticas definen la velocidad y posición.

Vemos entonces que Bug2 y Braitenberg, generarn sus trayectorias basándose en (x, y, θ) definidos por cada uno de estos modelos, ie, simulant su movimiento con modelos diferenciales



Cinemática de Robots Móviles

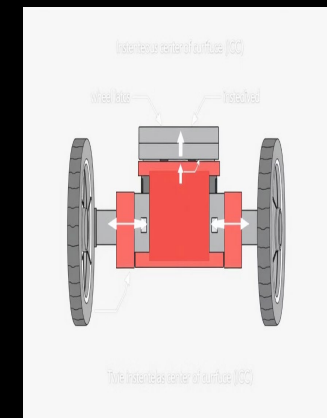
MODELOS CINEMATICOS

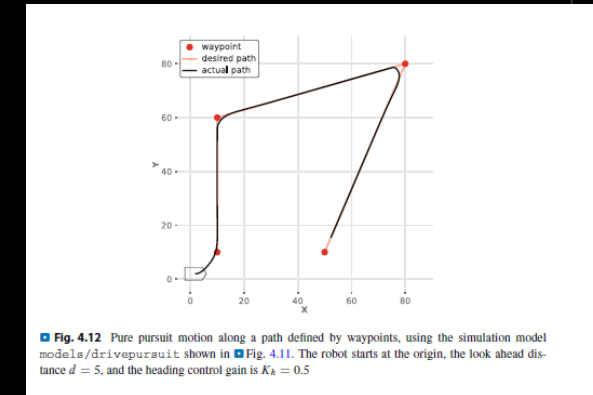
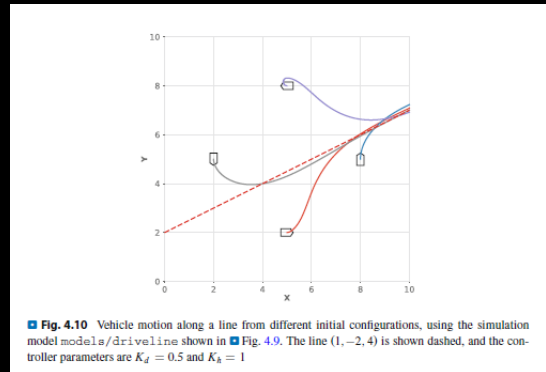
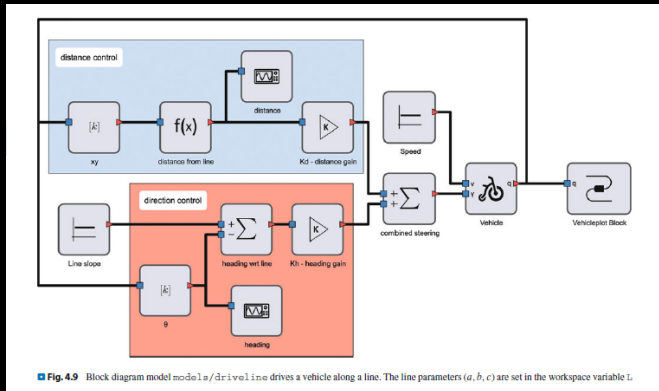
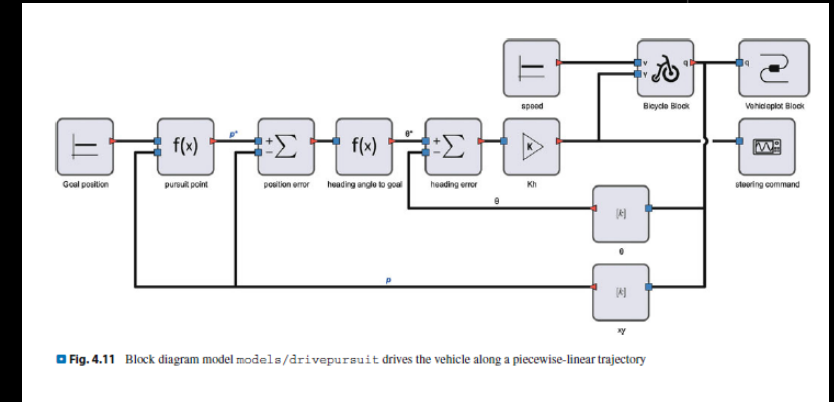
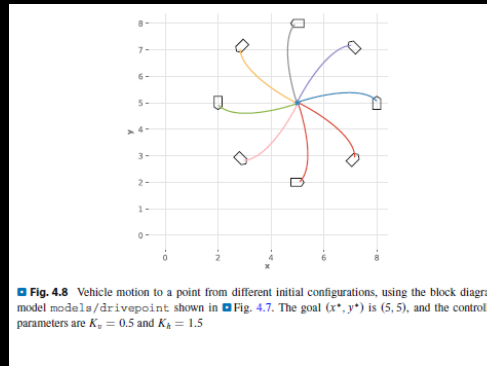
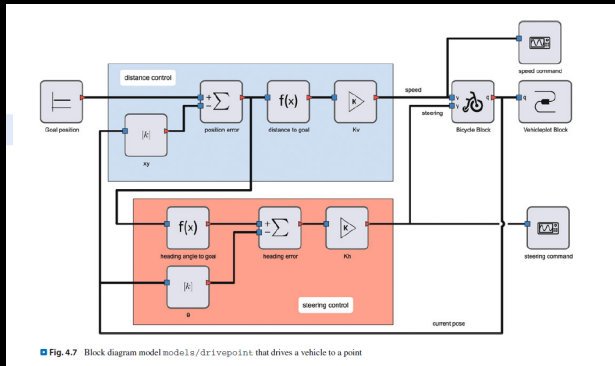
Ackermann : De igual manera tenemos los modelos que imitan el comportamiento de un auto, estos son ideales para ilustrar limitaciones de movimiento (no holonómicas) y su relevancia en la planificación de trayectorias.

Notamos que la presenta rueda delantera direccional y traseras motrices. Presenta un radio de giro.

A la hora de planificar trayectorias tenemos diferentes opciones tales como :

- Manejar a un punto
- Manejar a lo largo de una línea
- Manejar a lo largo de trayectorias
- Manejar a una configuración





Opciones de ruta

Modo	Descripción breve	Relevancia
Driving to a Point	Se busca alcanzar una posición deseada en el plano.	✓ Navegación reactiva y planificada.
Driving along a Line	Movimiento continuo sobre una línea recta definida.	Menos útil para entornos complejos.
Driving along a Path	Seguimiento de trayectorias suaves o curvas.	✓ Relevante para comparación con A*, PRM, celdas.
Driving to a Configuration	Alcanza un punto con orientación final deseada.	✓ Crítico para SLAM, docking o navegación precisa.

Opciones de ruta (vehículo tipo coche)

Cinemática de Robots Móviles

MODELOS CINEMATICOS

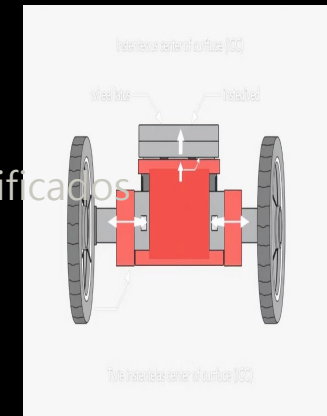
Esenciales para la navegación reactiva y la ejecución de movimientos.
Por ejemplo: diferencial y Ackermann.

Diferencial: presenta dos ruedas independientes y velocidad lineal.

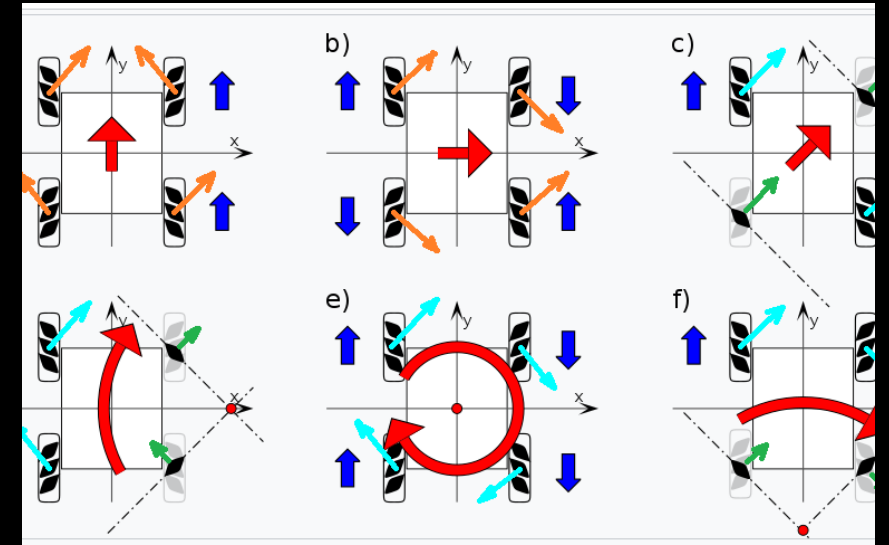
Ackermann : Rueda delantera direccional y traseras motrices. Presenta un radio de giro

Omnidireccional : Su mueve libremente por el espacio.

Los modelos cinemáticos, como los diferenciales para robots de dos ruedas y Ackermann para vehículos tipo coche son la base para ejecutar los movimientos planificados por los algoritmos de navegación.



Tipos de robots moviles Omnidireccional



Exploration: Mars Science Laboratory (MSL) rover, known as Curiosity, undergoing testing

Ppip físico:

Superposición vectorial de movimientos independientes.

Cada rueda contribuye con un vector de fuerza/velocidad en su eje de rodadura.

Omnidireccional

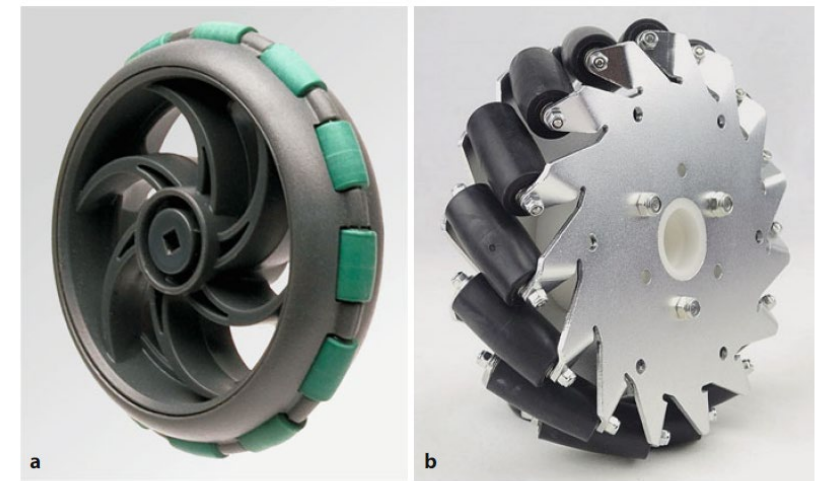


Fig. 4.18 Two types of omnidirectional wheel, note the different roller orientation. **a** Allows the wheel to *roll* sideways (courtesy VEX Robotics); **b** allows the wheel to *drive* sideways (courtesy of Nexus Robotics)

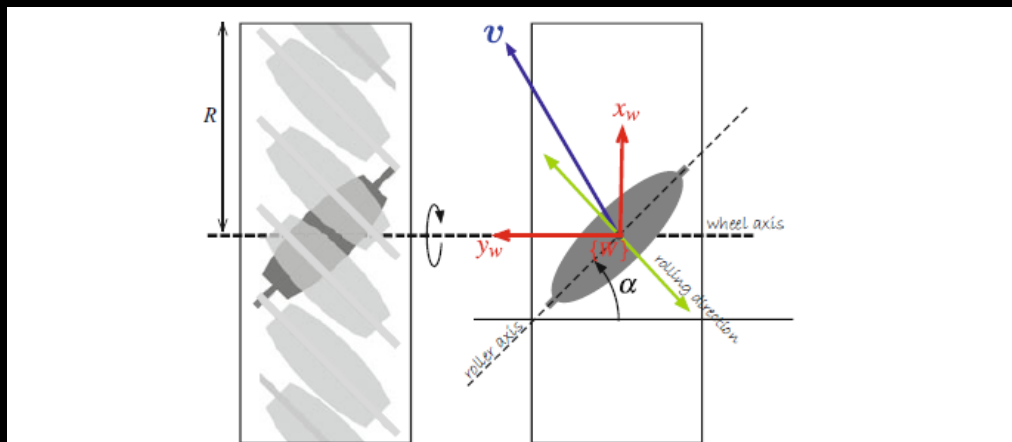


Fig. 4.19 Schematic of a mecanum wheel in plan view. The rollers are shown in gray and the dark gray roller is in contact with the ground. The green arrow indicates the rolling direction

Ruedas mecanum :

Rodillos pasivos a 45 grados centígrados.

Rodillos activos : Transmiten fuerza tangencial y permiten desplazamiento lateral

Finalmente, es clave tener presente que sin cinemática correcta, no se puede traducir la señal de control en movimiento real. **44**



a



b



c



d



ROBOTS AÉREOS.

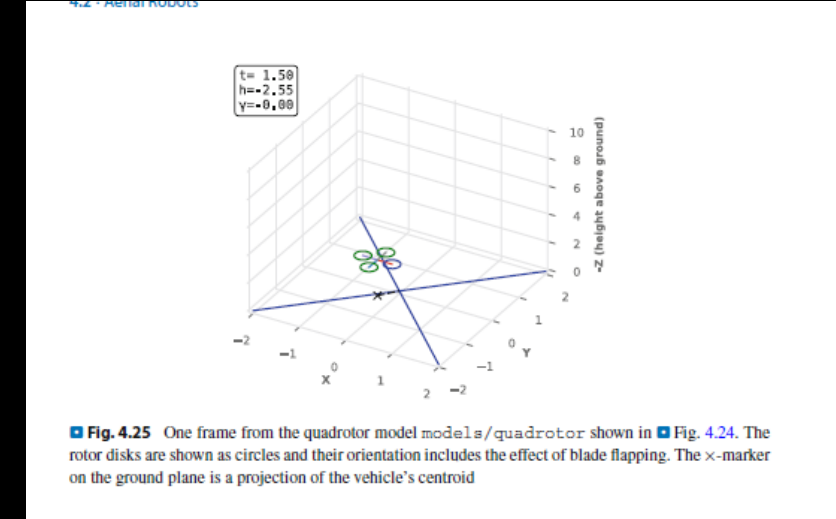
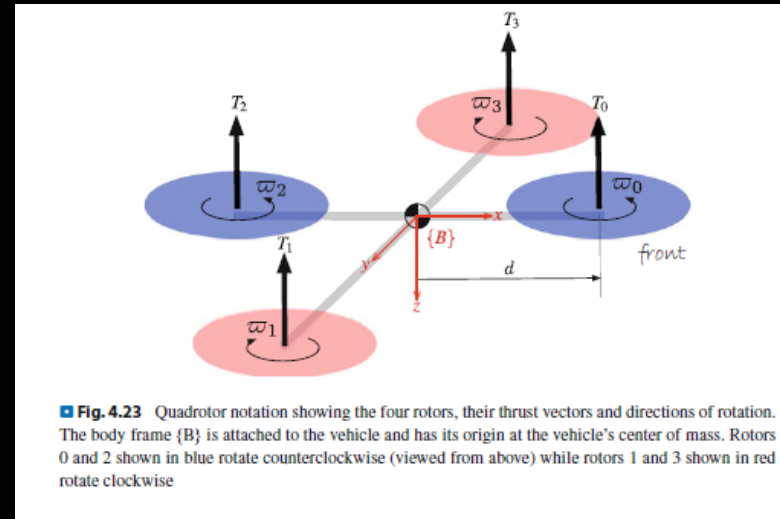
Cinemática de Robots Móviles

ROBOTS AÉREOS

- 4 rotores generan empuje individual
- con dirección y rotación específica
- Movimiento controlado por diferencias en empuje y dirección de giro

El marco de referencia del cuerpo es clave para describir orientación y pose

El centro de masa se proyecta sobre el plano del suelo (tracking visual)



- ROS2, nos permite implementar controladores específicos para cada modelo. Esto influye de manera positiva y directa en la navegación reactiva y planificada.

Navegación Basada en Mapas (Global)

Utiliza un mapa preexistente o construido para planificar rutas óptimas.

Considerando :

La más corta

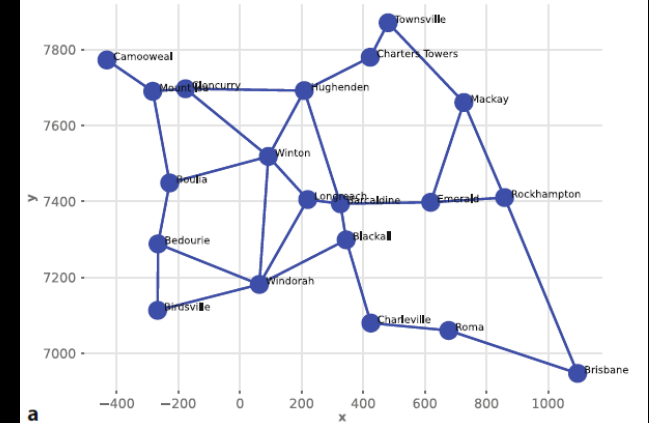
La más rápida

LA más Segura

Podemos encontrar:

• GOM, OCEI, ONOCE, RM, OCEMB

- Celdas: Cuadrículas de ocupación.
- PRM: Mapas de ruta probabilísticos.



Navegación Basada en Grafos

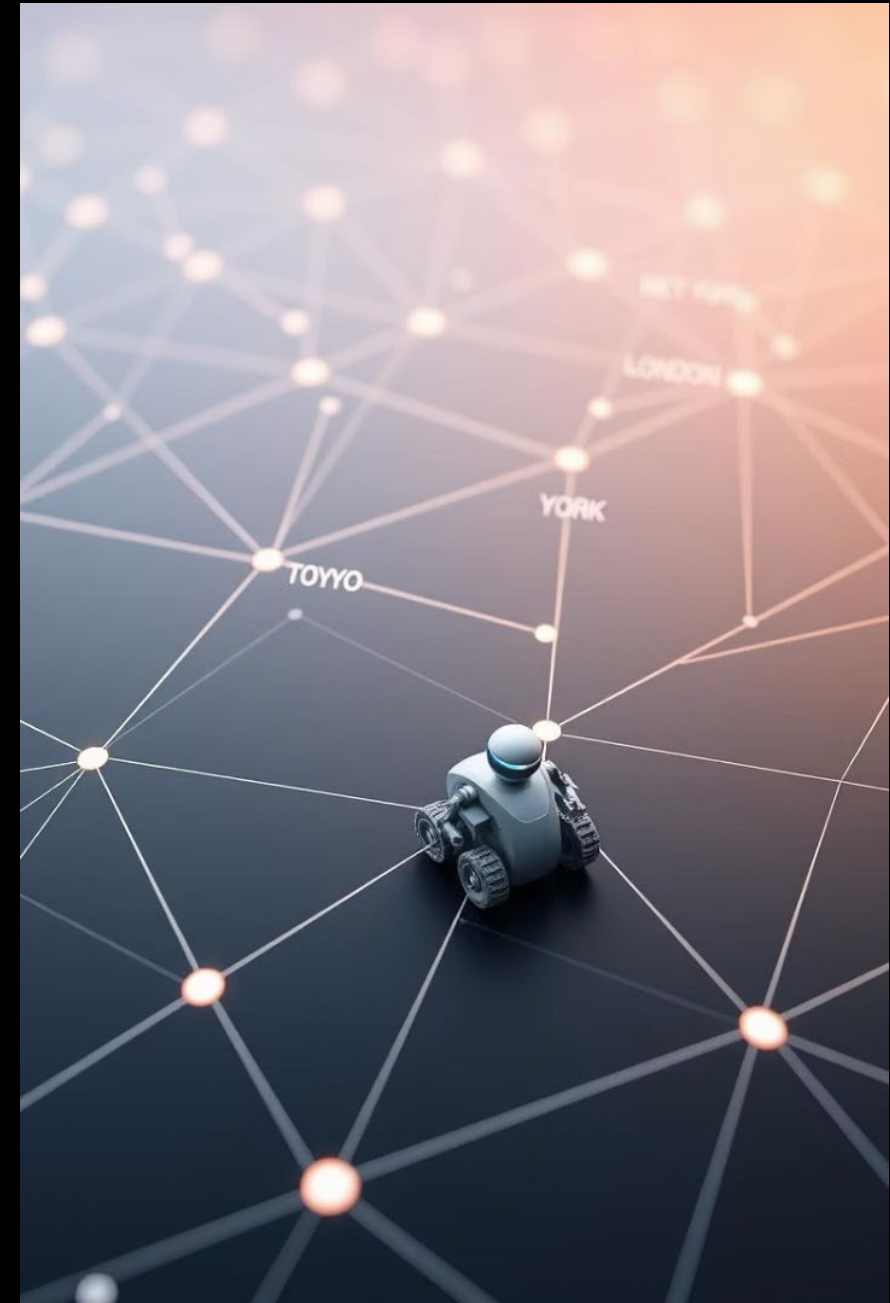
1

Representación

El entorno se modela como nodos (lugares) y aristas (caminos). Los costos
Los costos asociados indican distancia o tiempo

También es importante denotar que el uso de transformaciones homogéneas y cuaterniones permite definir marcos de referencia locales para la navegación.

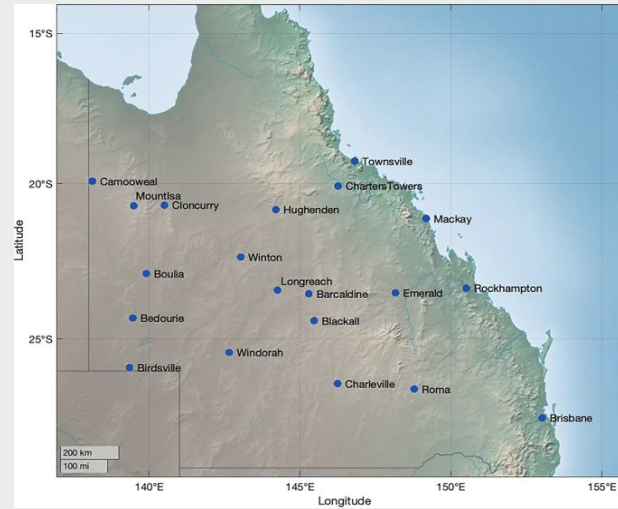
E.g, cuando se transforman las coordenadas del grafo a un marco del robot usando $SE(3)$



Navegación Basada en Grafos

Representación

Vemos también que, los vertices de los grafos representan lugares definidos por coordenadas espaciales.



BFS

Búsqueda en anchura: encuentra la ruta más corta en número de pasos.

UCS

Búsqueda de costo uniforme: garantiza la ruta de menor costo total.

A*

Búsqueda A estrellita: utiliza heurística para encontrar la ruta óptima de forma más eficiente.

5.3 Planning with a Graph-Based Map

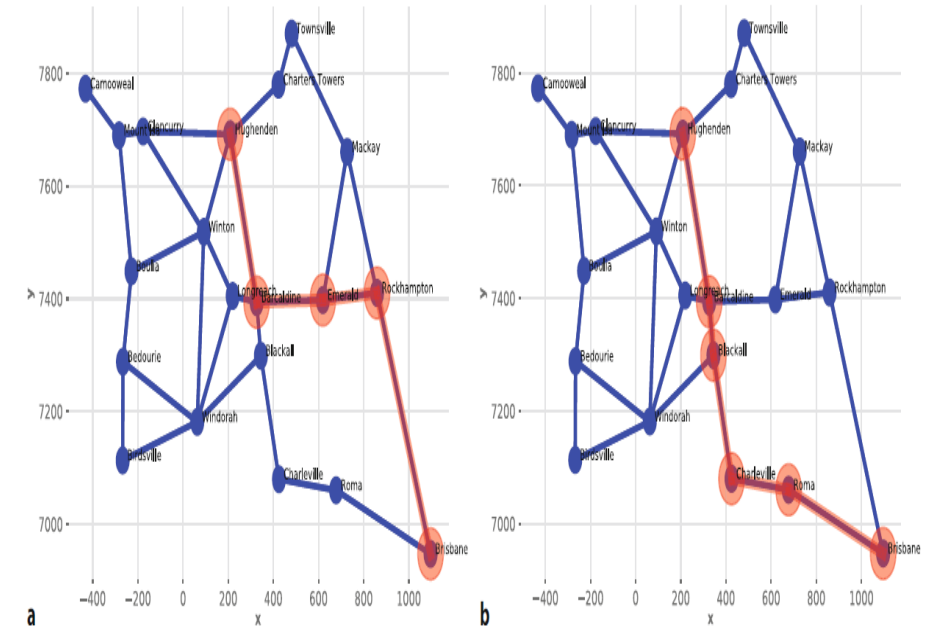


Fig. 5.8 Path planning results. a Breadth-first search (BFS) with a path length of 1759 km; b Uniform-cost search (UCS) and A* search with a path length of 1659 km

Algoritmos para búsqueda de rutas.

1. Breadth-First Search (BFS)

Prioriza caminos con **menos nodos**.

No considera la distancia real.

Devuelve una ruta rápida pero no óptima.

2. Uniform-Cost Search (UCS)

Expande el nodo con menor **costo acumulado ($g(v)$)**.

Devuelve la ruta de **menor distancia real**.

Usa una cola de prioridad y explora más nodos que BFS.

3. A* Search

Usa: $f(v) = g(v) + h^*(v)$

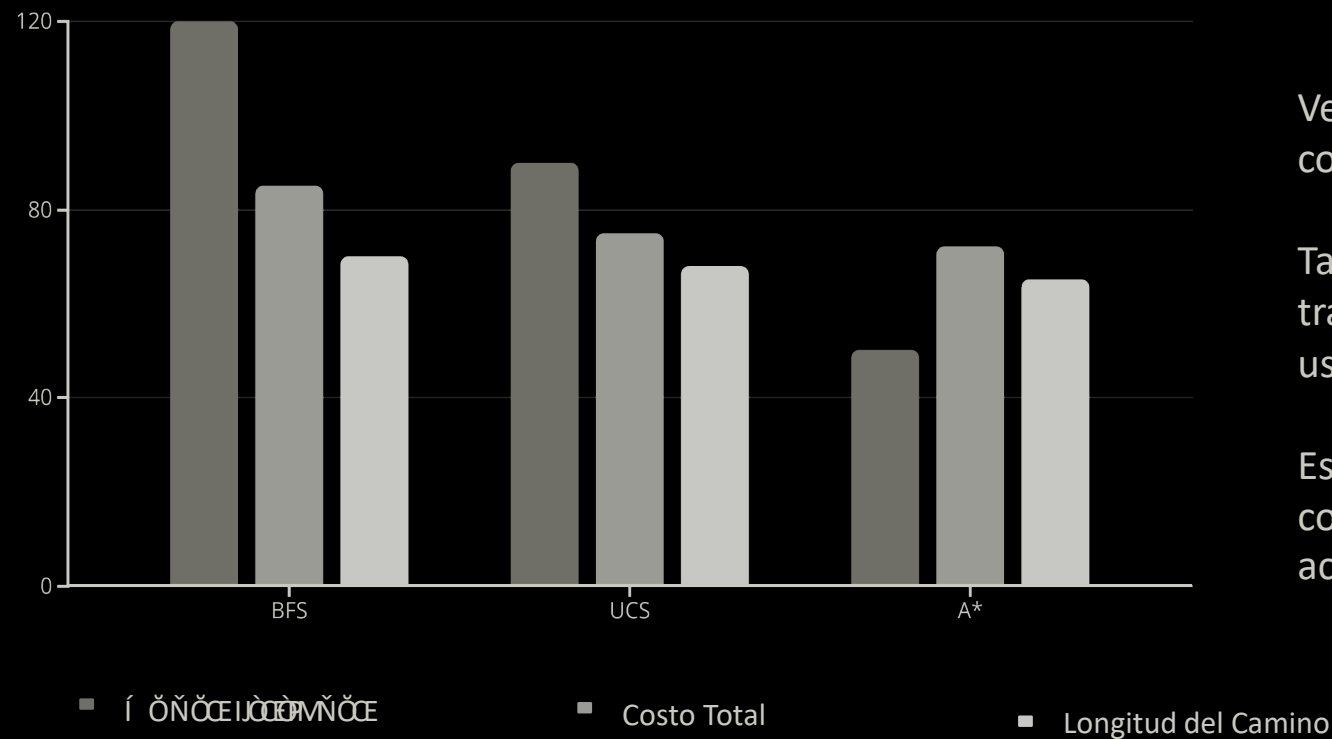
$g(v)$: costo acumulado.

$h^*(v)$: heurística admisible (ej: distancia Euclidiana).

Explora menos nodos que UCS.

Más eficiente si la heurística es buena.

Simulación y comparativa de algoritmos



Vemos que UCS y A* pueden considerar tiempo como costo (distancia / velocidad)

También está presente la planeacion de trayectorias, mediante interpolación entre nodos usando polinomios cúbicos/quínticos.

Es posible realizar un seguimiento temporal con controladores , mediante la velocidad y aceleración, e.g de motores.

La simulación muestra cómo A* visita menos nodos y encuentra caminos óptimos, superando a BFS y UCS en eficiencia para trayectorias complejas.

Relación con posición, orientación y tiempo

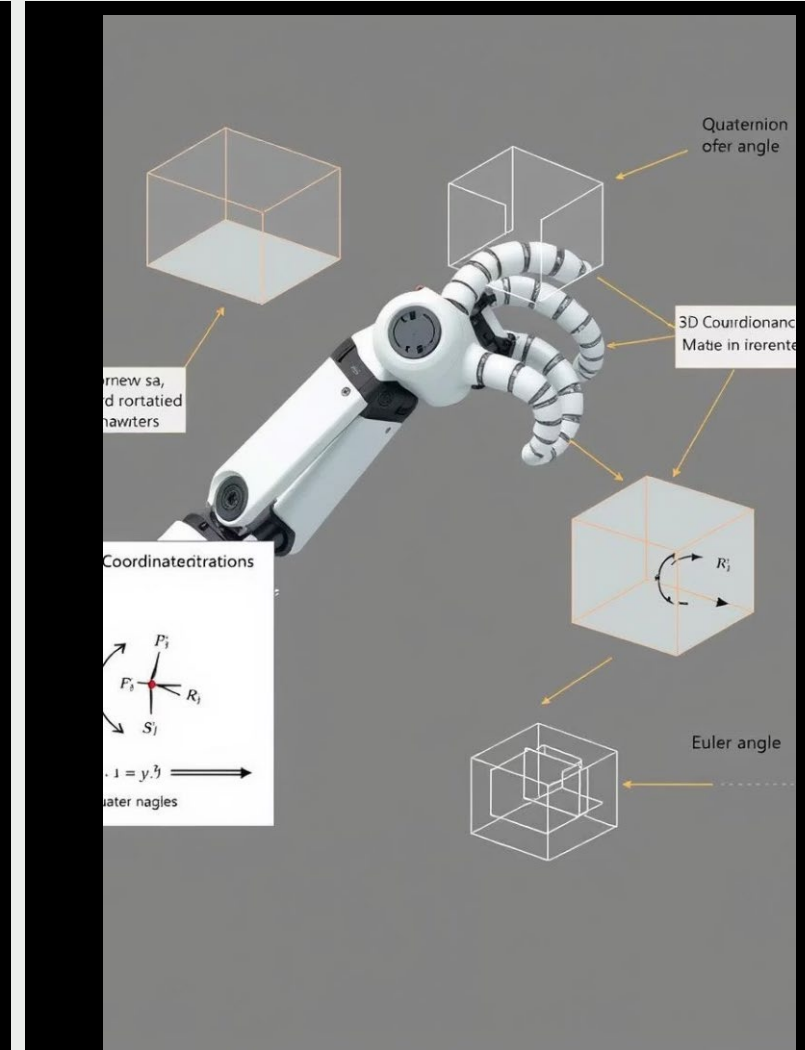
Los vértices de los grafos representan lugares definidos por coordenadas espaciales

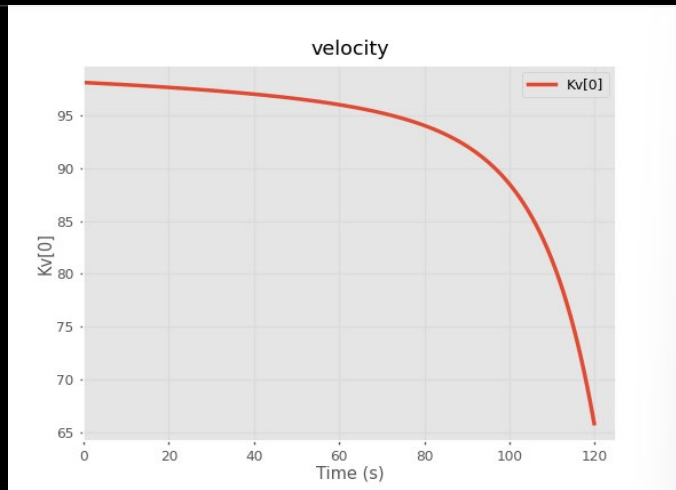
El uso de transformaciones homogéneas y cuaterniones permite definir marcos de referencia locales para navegación

Ejemplo: Transformar coordenadas del grafo a un marco del robot usando $SE(2)$ o $SE(3)$

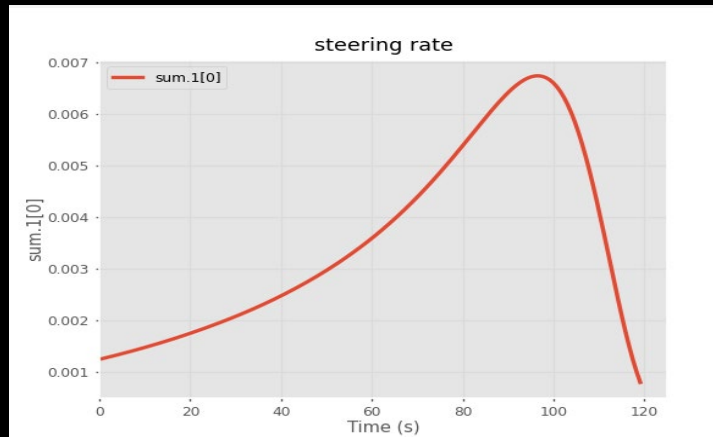
Estas herramientas nos permiten :

- *Cambiar entre marcos de referencia, por ejemplo de la base del robot a la herramienta
- * Integrar la orientación del robot en sistemas de navegación
- * Facilita la representación espacial en algoritmos de mapeo y localización





La gráfica de **velocity** (velocidad) muestra cómo varía la velocidad del vehículo Braitenberg a lo largo del tiempo durante la simulación.



La gráfica "**steering rate**" muestra el **comportamiento de la tasa de giro del vehículo Braitenberg (γ)** en función del tiempo durante la simulación. La tasa de giro (γ) es la velocidad angular con la que el vehículo ajusta su dirección de movimiento para alinearse con el gradiente del campo sensorial.

ESTOS CONCEPTOS SON IMPRESCINDIBLES PARA CONTROLADORES

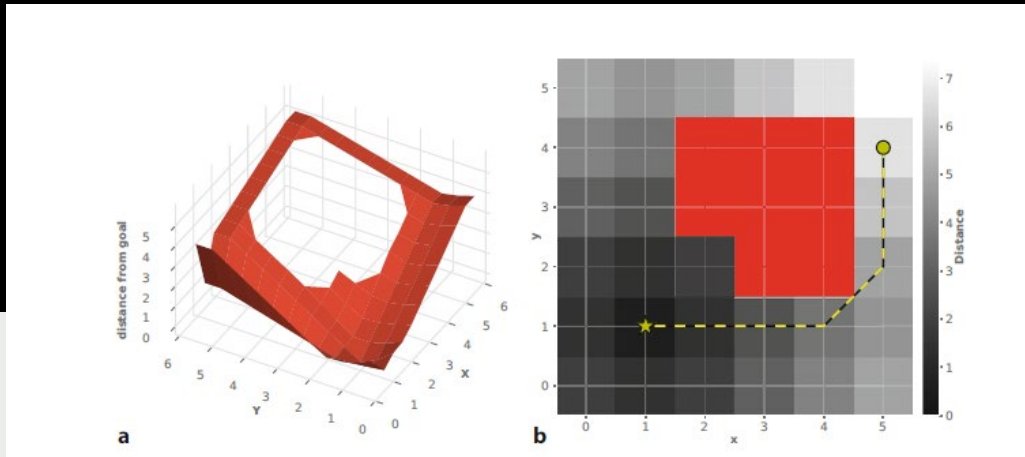
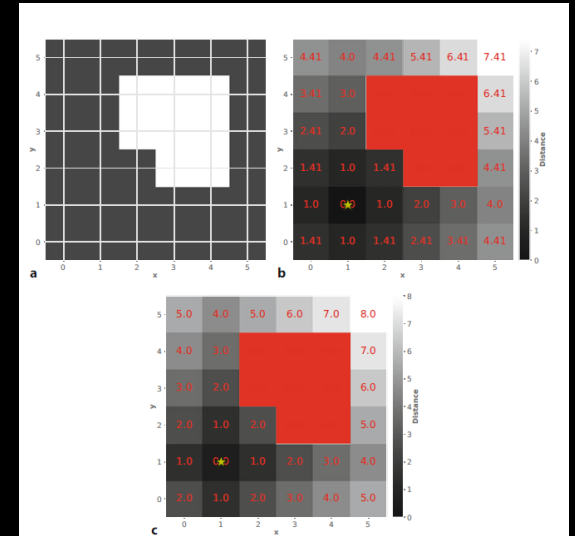
Relación con posición, orientación y tiempo

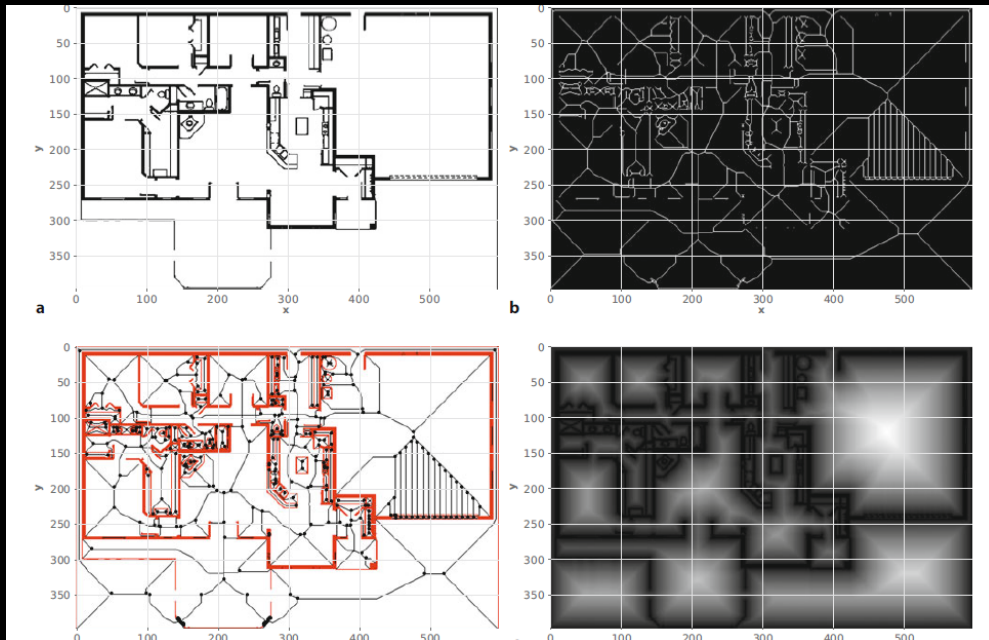
NAVEGACIÓN BASADA EN MAPAS

El entorno se modela como una cuadrícula

El entorno se modela como una cuadrícula. Cada celda indica si está libre (0) u ocupada (1).

- La transformada de distancia (Wavefront/ brushfire) genera un campo de costes.
- El robot sigue el gradiente descendente hacia el objetivo.





■ Idea General

- Separar en dos fases:
 1. **Planificación:** construir una red de caminos libres (grafo).
 2. **Consulta:** buscar una ruta entre un punto de inicio y uno final.

Por ejemplo, para los casos de Sekelton y Voronoi.

- Se genera una red de caminos usando:
 - **Skeletonization:** reduce el espacio libre a una red de píxeles centrales.
 - **Voronoi Diagram:** distancias a obstáculos generan líneas centrales equidistantes.
 - Finalmente se extraen intersecciones (junctions) y se procede a crear el grafo.



La calidad depende del muestreo.

- Eficiente para consultas rápidas de rutas.

Se realiza un muestreo aleatorio de puntos accesibles, conectados por líneas libres de obstáculos y de allí, se crea un grafo embebido.

Sin embargo, es importante tener en cuenta algunas consideraciones ;

- Puntos aleatorios no cubren todo el mapa: esto requiere realizar múltiples intentos
- Rutas pueden ser menos suaves.
- Puede fallar en pasajes estrechos.
- PRM o grafos solo funcionan si el modelo cinemático puede ejecutar las trayectorias generadas

Comparativa de Estrategias de Navegación

MÉTODO

- REACTIVA
- GRAFOS
- CELDAS
- PRM

VENTAJAS

- RAPIDA Y SIMPLE
- OPTIMA, HEURÍSTICA
- GLOBAL, PRECISA
- EFICIENTE EN CONSULTA

DESVENTAJAS

- SUBÓTIMA, SIN PREVISIÓN
- COSTOSA COMPUTACIONALMENTE
- MUY LENTA EN MAPAS GRANDES
- Dependiente del muestreo



Tipo de Robot

Modelo Cinemático

Navegación Típica

Compatibilidad Estrategias

Diferencial (2 ruedas)

Simétrico, puede girar en su eje

Interiores, entornos planos

✓ Reactiva (Bug), ✓ Grafos, ✓ Celdas



Ackermann (vehículos tipo auto)

Radio de giro limitado

Exteriores, caminos definidos

✓ Grafos, ✓ Celdas, ✗ Reactiva pura



Omnidireccional (Mecanum, holonómico)

Movimiento libre en XY

Espacios reducidos, laboratorios

✓ Reactiva, ✓ PRM



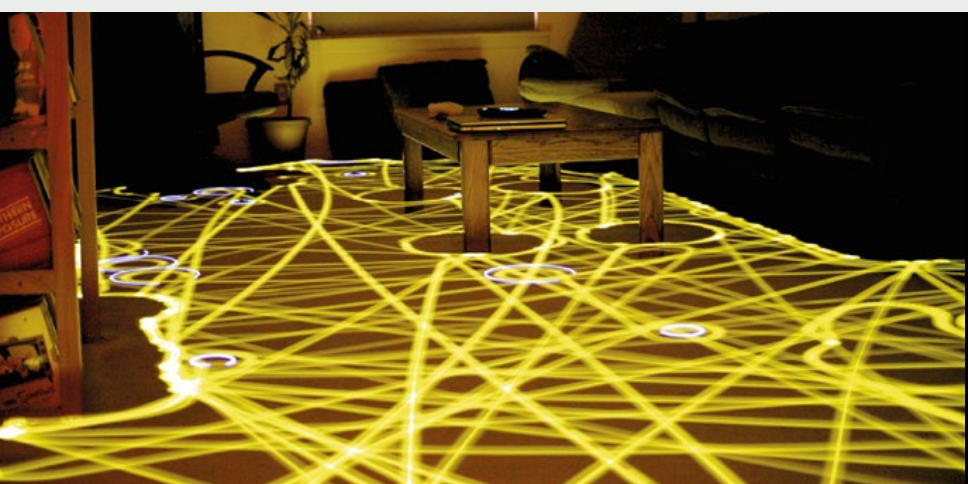
Aéreo (Drones)

6 DOF, control por orientación 3D

Navegación 3D, SLAM aéreo

✓ PRM, ✓ Grafos 3D, ✗ Celdas 2D

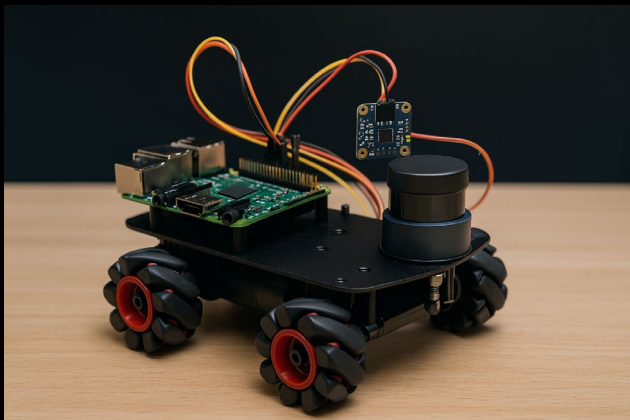
Comparativa de robots móviles



La navegación efectiva requiere la combinación de percepción, planificación y ejecución

Cada modelo tiene su aplicación ideal según contexto

El dominio de los conceptos de orientación, cinemática y tiempo permite adaptar soluciones a distintos entornos.



Desarrollar un robot móvil autónomo con capacidad de navegación inteligente en entornos interiores, utilizando:

Ruedas mecanum para movimiento omnidireccional.

Sensores inerciales (IMU) para estimación de orientación.

SLAM (Simultaneous Localization and Mapping) para construir un mapa del entorno y auto-localizarse.

Sistema embebido externo (e.g. Raspberry Pi, Jetson Nano) como unidad de procesamiento y control.

Sobre el proyecto :

Robot móvil autónomo con movimiento omnidireccional y control remoto

Elemento	Componente	ROS2 Nodo	Descripción
IMU	MPU6050 / BNO055	/imu	Publica orientación y aceleración
Ruedas Mecanum	4 Motores	/odom	Calcula odometría
SLAM	LIDAR + mapa (OPCIONAL)	/map, /tf	Genera mapa y localización
Control	Nodo embebido	/cmd_vel	Ejecuta comandos de movimiento

Sobre el proyecto

Desarrollo del Robot

Implementación de un robot autónomo omnidireccional con ruedas mecanum.

Integración de un módulo IMU para localización y navegación.

Sistemas de Control

Uso de la ALU de una Raspberry Pi 4 para protocolos de comunicación y control.
Implementación de filtros (Kalman y PDI) para mitigar el ruido en los sensores.

Prevención de Problemas

Incorporación de un sensor ultrasónico para evitar el *deadlocking* (bloqueos).

Entorno de Pruebas

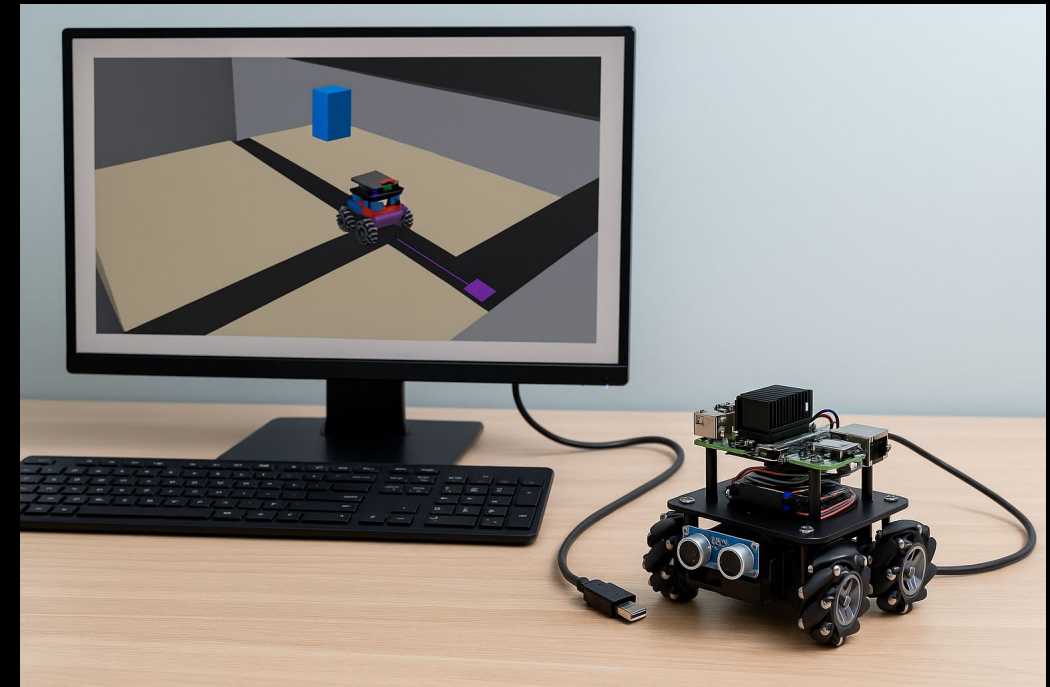
Realización de simulaciones básicas y pruebas en un entorno controlado.

Desarrollo iterativo con validaciones prácticas.

Resultados Esperados

Entrega de un prototipo funcional.

Compatibilidad con sistemas como ROS2 o PV (Python-Virtual) para manejo avanzado.



Alcances y aspiraciones por FASES:

FASE 1 : SIMULACIONES

FASE 2: PRUEBA DE SENSORES

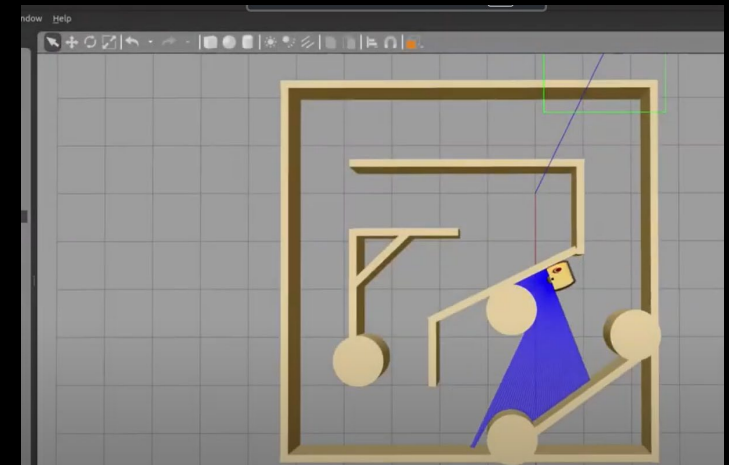
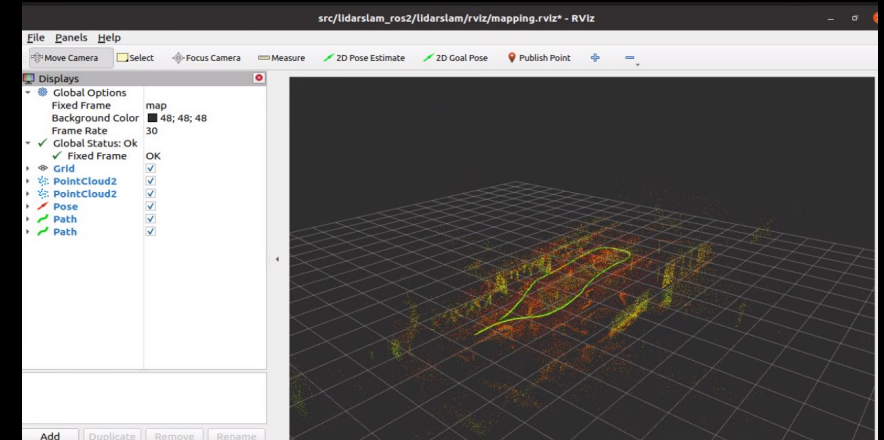
FASE 3 : PROTOTITPO MECANUM

FASE 4 : PROTOCOLOS DE COMUNICACION

ROS2

Finalmente una visualización de lo que nos permite el entorno de ROS2 en linux

Este nos permite implementar controladores específicos para cada modelo. Esto influye de manera positiva y directa en la navegación reactiva y planificada.



GRACIAS